JENKINS CI/CD PIPELINE & TERRAFORM INFRASTRUCTURE AS CODE ASSIGNMENT

Automated Build, Test and Deployment of the TaskFlow Web Application using Jenkins and Terraform

Name : Abhay Maurya

PRN : 202301040187

Course : CND (MDM)

GitHub Repository : https://github.com/Abhxym/taskflow-cicd

ABSTRACT

Continuous Integration and Continuous Delivery (CI/CD) automate the path from a code change to a running application, and Infrastructure as Code (IaC) makes the servers and containers that run the application reproducible. This assignment combines both practices for a web application called TaskFlow, a task board built with Node.js.

A Jenkins server runs in Docker and is fully configured as code (plugins, admin user and the pipeline job). The job reads its Jenkinsfile from the GitHub repository and checks GitHub for new commits every two minutes. The pipeline has ten stages: checkout, unit tests with a JUnit report, Docker image build, container test, Terraform validation, deployment to staging, a smoke test, a manual approval, deployment to production and a final smoke test.

Terraform (v1.17.1, Docker provider) creates the infrastructure for each environment from a reusable module: a network, a data volume, a number of application containers with health checks and memory limits, and an Nginx load balancer whose configuration is generated from the list of containers. Staging runs two replicas and production runs three. Each environment uses its own Terraform workspace.

The pipeline was triggered automatically by commits pushed to GitHub and completed successfully. A Terraform plan in the pipeline log showed that the data volume was being recreated on every deployment; this was fixed and a later build confirmed that saved tasks survive a deployment.

Keywords: DevOps, CI/CD, Jenkins, Jenkinsfile, Pipeline as Code, Terraform, Infrastructure as Code, Docker, GitHub, Configuration as Code.

1. INTRODUCTION

1.1 Background

In many teams a new version of an application is still built, tested and copied to the server by hand. Each step depends on the person doing it, so mistakes such as skipping tests, deploying the wrong version or configuring a server differently are common. Continuous Integration solves the first part: every change pushed to the repository is built and tested automatically. Continuous Delivery and Deployment solve the second part: a tested build is deployed to the target environments by the pipeline, with a manual approval where needed.

Jenkins is an open source automation server that is widely used for CI/CD. A Jenkins pipeline is written in a file named Jenkinsfile that is stored together with the application code, so the pipeline itself is version controlled.

Terraform is an Infrastructure as Code tool. Instead of creating servers, networks and containers manually, the required infrastructure is written in configuration files. Terraform compares this desired

state with the real infrastructure, shows a plan and then makes only the required changes.

1.2 Problem Statement

TaskFlow needs a staging environment for testing and a production environment for users. Without automation, every release requires running the tests, building an image, stopping the old containers, starting new ones with the correct settings and checking that the site works, once for each environment. This process is slow and error prone, and the two environments can easily drift apart.

The aim of this assignment is to automate the whole release process with a Jenkins pipeline and to describe the environments with Terraform, so that a push to GitHub results in a tested, deployed and verified release.

1.3 Objectives

1. To develop a web application (TaskFlow) with automated unit and API tests.
2. To install and configure Jenkins with the required plugins, using Configuration as Code.
3. To create a Jenkins pipeline job integrated with the GitHub repository (Pipeline script from SCM, Poll SCM trigger).
4. To write a Jenkinsfile covering source checkout, testing, image build, container testing and deployment.
5. To write a structured and reusable Terraform configuration with a module, variables, outputs and per-environment variable files.
6. To provision staging and production environments with Terraform from the pipeline, with a manual approval before production.
7. To demonstrate the pipeline running automatically after a commit and to verify the deployed application.

1.4 Scope of the Project

- A Node.js task board application with a REST API and a web interface.
- Jenkins LTS running in Docker, configured with Configuration as Code and Job DSL.
- A declarative Jenkinsfile with ten stages, post actions, parameters and an approval step.
- Terraform with the kreuzwerker/docker provider, a reusable webapp module and three environments (dev, staging, production).
- Repository integration with GitHub using Poll SCM.

The infrastructure is created as Docker resources on the local machine so that no cloud account or paid service is needed. The same Terraform structure can be used with a cloud provider by changing the provider and resources. GitHub webhooks were not used because the local Jenkins server is not reachable from the internet; Poll SCM is used instead.

2. APPLICATION OVERVIEW

2.1 TaskFlow

TaskFlow is a task board for a small team. Tasks are shown in four columns (Backlog, In progress, Review, Done) and can be moved between columns with drag and drop. Each task has a title, description, tag, priority, due date and assignee. The board shows sprint statistics and a progress bar, and it supports search, filtering by tag and keyboard shortcuts.

The side panel shows the environment, version, Jenkins build number, Git commit and container instance of the running application. These values are passed in by Jenkins and Terraform, which makes it easy to check which build is deployed in each environment.

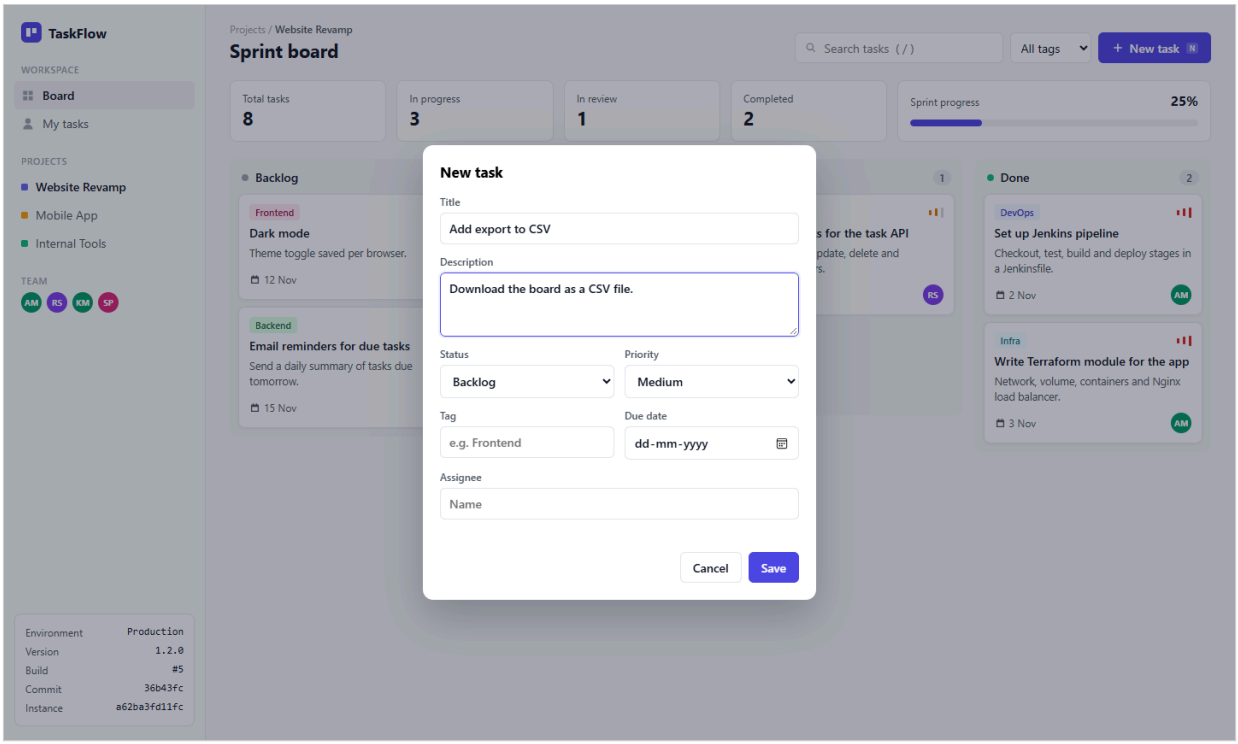


Figure 1: TaskFlow board with the New task dialog (production, Jenkins build #5)

2.2 Application Structure and API

The server (app/server.js) uses only the built-in modules of Node.js. Tasks are stored in a JSON file in the folder given by DATA_DIR, which is a Docker volume in the deployed environments.

Method and path	Purpose
GET /api/health	Health check used by Docker, Terraform and the pipeline
GET /api/info	Version, build number, commit, environment and instance
GET /api/tasks	List all tasks
POST /api/tasks	Create a task (validated)
PATCH /api/tasks/:id	Update a task, for example its status
DELETE /api/tasks/:id	Delete a task

2.3 Automated Tests

There are 14 tests written with the built-in Node.js test runner. test/store.test.js checks the task store and input validation (required title, allowed status and priority values, default values, saving to disk). test/api.test.js starts the server on a random port and checks the health and info endpoints, creating, updating and deleting a task, the web page and unknown routes.

3. CI/CD WORKFLOW DESIGN

3.1 CI/CD Concepts Used

- **Continuous Integration:** every commit on the main branch is checked out, tested and built automatically.
- **Continuous Delivery:** every successful build is deployed to staging automatically and is ready for production.

- **Approval gate:** production deployment waits for a person to approve it in Jenkins.
- **Pipeline as Code:** the pipeline is defined in the Jenkinsfile in the repository.
- **Infrastructure as Code:** environments are created by Terraform, never by hand.
- **Immutable artifacts:** each build produces an image tagged with the build number; the same image is deployed to staging and production.
- **Fail fast:** if any stage fails, the pipeline stops and nothing is deployed.

3.2 Pipeline Diagram

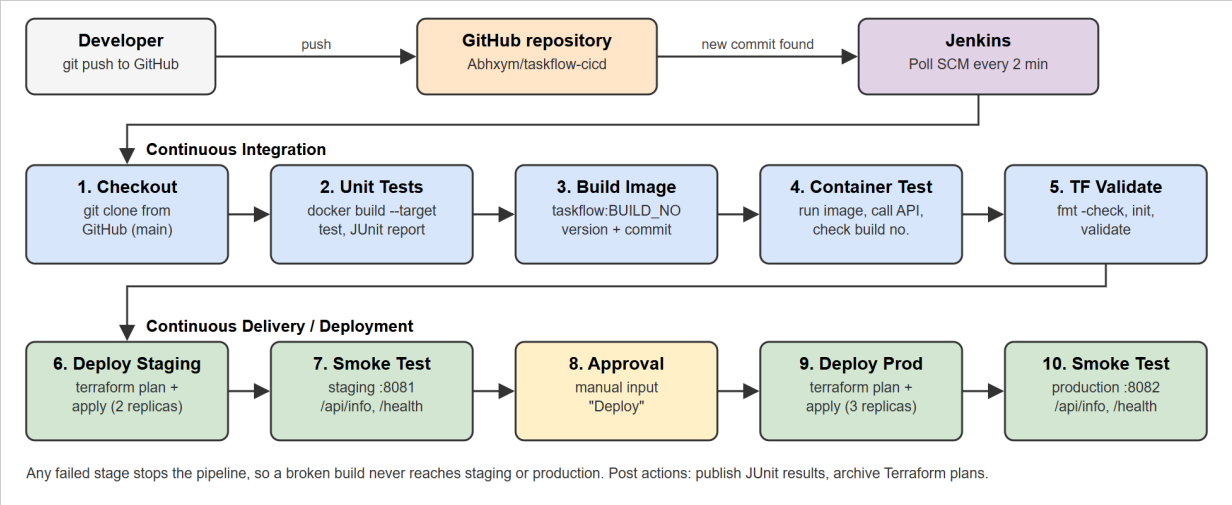


Figure 2: CI/CD pipeline from git push to production

3.3 Pipeline Stages

No.	Stage	What it does	Result if it fails
1	Checkout	Clones the commit from GitHub, reads version and short commit id	Build stops
2	Unit Tests	Builds the test stage of the Dockerfile (runs 14 tests), copies out the JUnit report	Build marked failed, report shows the failing test
3	Build Image	Builds taskflow:BUILD_NUMBER with version, build and commit as build arguments	Build stops
4	Container Test	Starts the new image, calls the API, checks the build number, creates a task and checks validation	Image is not deployed
5	Terraform Validate	terraform fmt -check, init and validate	Nothing is deployed
6	Deploy to Staging	terraform plan and apply in the staging workspace	Production is not touched
7	Smoke Test Staging	Checks that staging serves this build through the load balancer	Production is not touched
8	Approval	Waits for a person to click Deploy (15 minute timeout)	Release stops at staging
9	Deploy to Production	terraform plan and apply in the production workspace	Build failed
10	Smoke Test Production	Checks that production serves this build	Build failed

3.4 Tools and Technologies

Technology	Purpose
Jenkins 2.541.3 LTS (JDK 17)	CI/CD server running the pipeline
Jenkins plugins	Pipeline, Git, GitHub, JUnit, Stage View, Pipeline Graph View, Configuration as Code, Job DSL, Timestamper, AnsiColor
Jenkinsfile (declarative)	Pipeline as code
Git and GitHub	Source code repository and trigger for the pipeline
Docker / Docker Desktop	Builds images and runs Jenkins, staging and production
Terraform 1.17.1	Infrastructure as Code
kreuzwerker/docker provider 3.6	Lets Terraform create Docker networks, volumes, images and containers
Nginx 1.27	Load balancer in each environment
Node.js 22 (Alpine image)	Application runtime and test runner
Windows PowerShell	Running Terraform by hand for the dev environment

4. SYSTEM ARCHITECTURE

4.1 Architecture Overview

Everything runs on Docker Desktop. Jenkins runs in a container and uses the Docker engine of the host through the mounted Docker socket, so the images it builds and the containers that Terraform creates appear on the host. Jenkins clones the code from GitHub, builds and tests the image, and then runs Terraform, which creates one set of resources for staging and another for production.

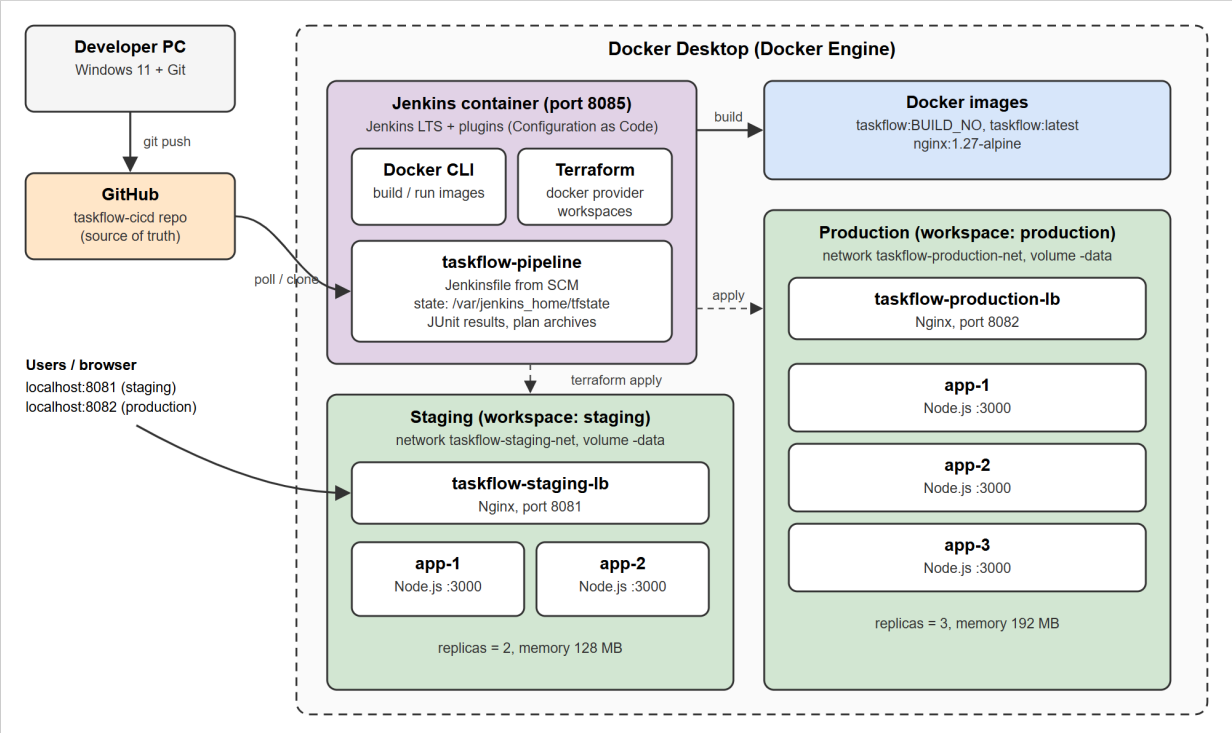


Figure 3: System architecture

4.2 Environments

Environment	Created by	Replicas	Memory per container	URL
dev	Terraform run by hand (PowerShell)	1	128 MB	http://localhost:8083
staging	Jenkins pipeline (automatic)	2	128 MB	http://localhost:8081
production	Jenkins pipeline (after approval)	3	192 MB	http://localhost:8082

5. PROJECT STRUCTURE

```
taskflow-cicd/
├── Jenkinsfile                 CI/CD pipeline
├── Dockerfile                  multi-stage: test stage + runtime image
├── package.json
├── app/                        server.js, store.js
├── public/                     index.html, styles.css, app.js
├── test/                       store.test.js, api.test.js
├── jenkins/
│   ├── Dockerfile              Jenkins LTS + Docker CLI + Terraform + plugins
│   ├── plugins.txt
│   ├── casc.yaml               users, security and the pipeline job
│   ├── docker-compose.yml
│   └── .env.example            (real .env with the password is not committed)
├── terraform/
│   ├── versions.tf  providers.tf  variables.tf  main.tf  outputs.tf
│   ├── .terraform.lock.hcl
│   ├── environments/           dev.tfvars, staging.tfvars, production.tfvars
│   └── modules/webapp/         main.tf, variables.tf, outputs.tf, versions.tf,
│                               templates/nginx.conf.tftpl
└── docs/screenshots/           execution evidence
```

6. JENKINS CONFIGURATION

6.1 Jenkins Server Image

Jenkins runs from a custom image based on the official jenkins/jenkins:lts-jdk17 image. The Docker CLI and Terraform are added so the pipeline can build images and create infrastructure. The plugins are installed during the image build from plugins.txt, and the setup wizard is turned off because everything is configured as code.

File: jenkins/Dockerfile

```
# Jenkins controller for the TaskFlow pipeline.
# Adds the Docker CLI and Terraform so pipeline stages can build images
# and provision infrastructure. Plugins and the job are set up as code.
FROM jenkins/jenkins:lts-jdk17

USER root

RUN apt-get update && \
    apt-get install -y --no-install-recommends curl unzip ca-certificates git && \
    rm -rf /var/lib/apt/lists/*

# Docker CLI only (the Docker engine of the host is used through the socket)
ARG DOCKER_VERSION=27.5.1
RUN curl -fsSL "https://download.docker.com/linux/static/stable/x86_64/docker-${DOCKER_VERSION}.tgz" \
      | tar -xz -C /tmp && mv /tmp/docker/docker /usr/local/bin/docker && rm -rf /tmp/docker

# Terraform CLI
ARG TERRAFORM_VERSION=1.17.1
RUN curl -fsSL
"https://releases.hashicorp.com/terraform/${TERRAFORM_VERSION}/terraform_${TERRAFORM_VERSION}_linux_amd64.zip"
\
      -o /tmp/terraform.zip && unzip -q /tmp/terraform.zip terraform -d /usr/local/bin && rm
/tmp/terraform.zip

# Plugins
COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
```

```
RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt

# Configuration as Code (users, security, the pipeline job)
COPY casc.yaml /usr/share/jenkins/ref/casc.yaml
ENV CASC_JENKINS_CONFIG=/usr/share/jenkins/ref/casc.yaml \
    JAVA_OPTS="-Djenkins.install.runSetupWizard=false"
```

File: jenkins/plugins.txt

```
workflow-aggregator
git
github
junit
pipeline-stage-view
pipeline-graph-view
configuration-as-code
job-dsl
timestamper
ansicolor
ws-cleanup
```

6.2 Configuration as Code

The Configuration as Code plugin reads casc.yaml when Jenkins starts. It creates the admin user (the password comes from the jenkins/.env file, which is not committed), allows anonymous users to view builds but not change anything, sets the Jenkins URL and creates the pipeline job with Job DSL. The job uses **Pipeline script from SCM**, so the Jenkinsfile is read from the main branch of the GitHub repository, and a **Poll SCM** trigger (H/2 * * * *) checks GitHub for new commits every two minutes.

File: jenkins/casc.yaml

```
# Jenkins Configuration as Code
# Values like ${JENKINS_ADMIN_PASSWORD} come from jenkins/.env (not committed).
jenkins:
  systemMessage: "TaskFlow CI/CD server - builds, tests and deploys TaskFlow with Docker and Terraform."
  numExecutors: 2
  mode: NORMAL
  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: "${JENKINS_ADMIN_USER}"
          name: "Abhay Maurya"
          password: "${JENKINS_ADMIN_PASSWORD}"
  authorizationStrategy:
    loggedInUsersCanDoAnything:
      allowAnonymousRead: true

unclassified:
  location:
    url: "http://localhost:8085/"

jobs:
  - script: |
      pipelineJob('taskflow-pipeline') {
        displayName('TaskFlow CI/CD Pipeline')
        description('Checkout from GitHub, unit tests, Docker build, container tests, Terraform deploy to
staging, approval, deploy to production.')
        properties {
          githubProjectUrl('${TASKFLOW_REPO_URL}')
          pipelineTriggers {
            triggers {
              pollSCM {
                scmpoll_spec('H/2 * * * *')
              }
            }
          }
        }
        definition {
          cpsScm {
            scm {
              git {
                remote {
```

```
                url('${TASKFLOW_REPO_URL}')
              }
              branch('*/main')
            }
          }
          scriptPath('Jenkinsfile')
          lightweight(true)
        }
      }
    }
```

The Jenkins container gets the Docker socket of the host and port 8085:

File: jenkins/docker-compose.yml

```
# Starts the Jenkins server:  docker compose -f jenkins/docker-compose.yml up -d --build
services:
  jenkins:
    build: .
    image: taskflow-jenkins:lts
    container_name: jenkins
    # root is needed to use the Docker socket of the host (lab setup only)
    user: root
    env_file: .env
    ports:
      - "8085:8080"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    networks: [ci]
    extra_hosts:
      - "host.docker.internal:host-gateway"
    restart: unless-stopped

volumes:
  jenkins_home:

networks:
  ci:
    name: ci-net
```

6.3 Starting Jenkins

docker compose -f jenkins/docker-compose.yml up -d --build

Jenkins started at http://localhost:8085 with the job already created and no manual setup.

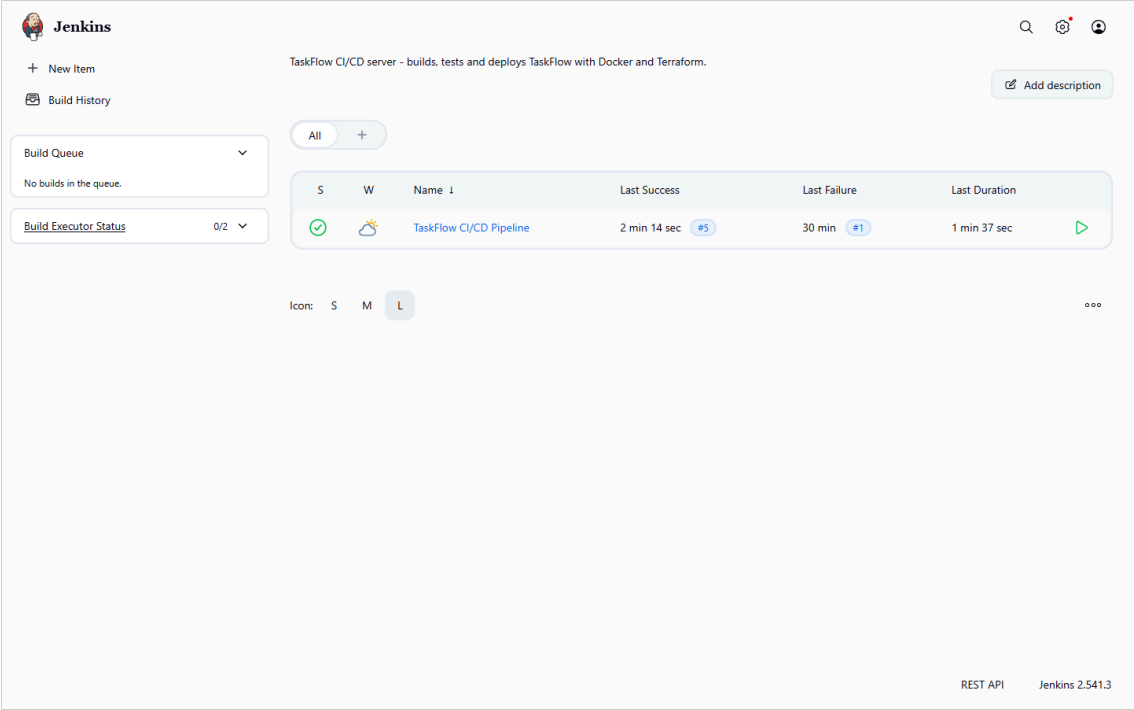


Figure 4: Jenkins dashboard with the TaskFlow CI/CD Pipeline job

6.4 Job Configuration and Repository Integration

The job configuration created by Job DSL shows the GitHub project, the Poll SCM trigger and the Pipeline script from SCM definition pointing at the repository and the Jenkinsfile:

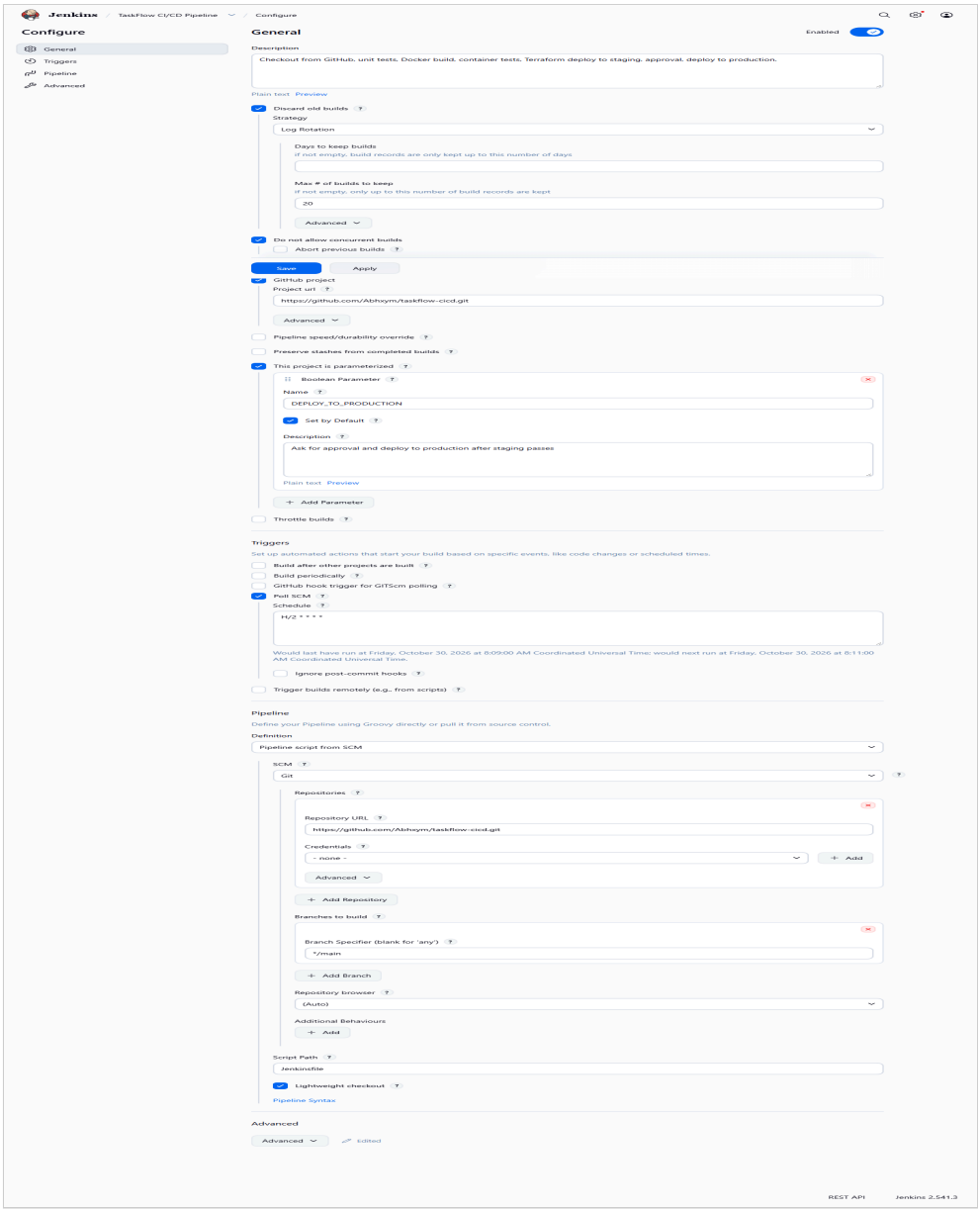


Figure 5: Job configuration: GitHub project, Poll SCM trigger and Pipeline from SCM

The Git polling log shows Jenkins finding a new commit on the main branch and starting a build:

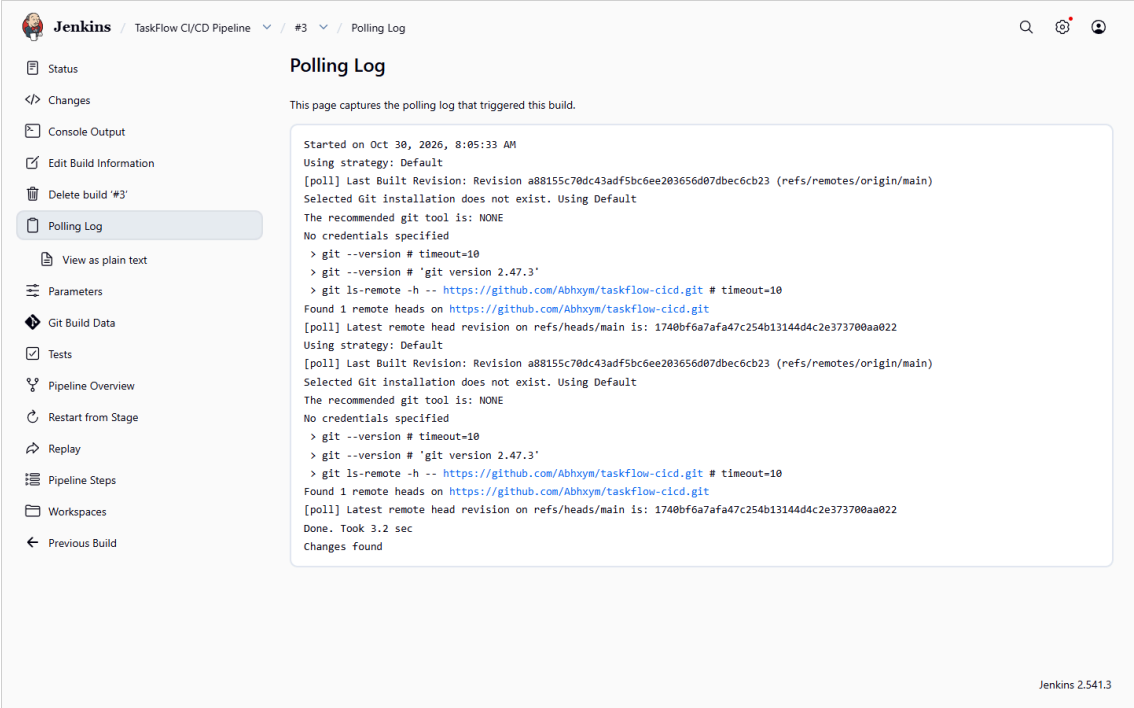


Figure 6: Polling log for build #3: a change was found in the GitHub repository

7. JENKINSFILE (CI/CD PIPELINE)

The pipeline is declarative. The options block adds timestamps, coloured output, a 30 minute timeout, old build cleanup and prevents two builds from running at the same time. The DEPLOY_TO_PRODUCTION parameter allows a build to stop after staging. Two small functions, deploy() and smokeTest(), are used for both environments so the same steps are not written twice.

File: Jenkinsfile

```
// TaskFlow CI/CD pipeline
// Checkout -> Unit tests -> Build image -> Container test -> Terraform validate
// -> Deploy staging -> Smoke test -> Approval -> Deploy production -> Smoke test

pipeline {
    agent any

    options {
        skipDefaultCheckout(true)    // the Checkout stage does it once
        timestamps()
        ansiColor('xterm')
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '20'))
        timeout(time: 30, unit: 'MINUTES')
    }

    parameters {
        booleanParam(name: 'DEPLOY_TO_PRODUCTION', defaultValue: true,
                     description: 'Ask for approval and deploy to production after staging passes')
    }

    environment {
        APP_NAME            = 'taskflow'
        IMAGE_TAG           = "${env.BUILD_NUMBER}"
        CI_NETWORK          = 'ci-net'
        TF_IN_AUTOMATION    = '1'
        TF_INPUT            = '0'
        TF_PLUGIN_CACHE_DIR = '/var/jenkins_home/.terraform.d/plugin-cache'
        TF_STATE_DIR        = '/var/jenkins_home/tfstate/taskflow'
    }

    stages {
```

```
        stage('Checkout') {
            steps {
                checkout scm
                script {
                    env.GIT_SHORT   = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
                    env.APP_VERSION = sh(script: "grep -m1 '\"version\"' package.json | cut -d'\"' -f4",
returnStdout: true).trim()
                    currentBuild.description = "v${env.APP_VERSION} (${env.GIT_SHORT})"
                }
                sh 'git log -1 --pretty="Commit %h by %an: %s"'
                echo "Building ${APP_NAME} version ${env.APP_VERSION}, build #${env.BUILD_NUMBER}"
            }
        }

        stage('Unit Tests') {
            steps {
                // tests run inside the "test" stage of the Dockerfile
                sh 'docker build --target test -t ${APP_NAME}-test:${IMAGE_TAG} .'
                sh '''
                    docker create --name ${APP_NAME}-results-${IMAGE_TAG} ${APP_NAME}-test:${IMAGE_TAG}
                    docker cp ${APP_NAME}-results-${IMAGE_TAG}:/src/test-results.xml test-results.xml
                    docker rm ${APP_NAME}-results-${IMAGE_TAG}
                '''
            }
            post {
                always {
                    junit 'test-results.xml'
                }
            }
        }

        stage('Build Image') {
            steps {
                sh '''
                    docker build \
                      --build-arg APP_VERSION=${APP_VERSION} \
                      --build-arg BUILD_NUMBER=${BUILD_NUMBER} \
                      --build-arg GIT_COMMIT=${GIT_SHORT} \
                      -t ${APP_NAME}:${IMAGE_TAG} -t ${APP_NAME}:latest .
                '''
                sh 'docker image ls ${APP_NAME} --format "table
{{.Repository}}\\t{{.Tag}}\\t{{.ID}}\\t{{.Size}}" | head -5'
            }
        }

        stage('Container Test') {
            steps {
                sh '''
                    docker run -d --name ${APP_NAME}-ci-${IMAGE_TAG} --network ${CI_NETWORK}
${APP_NAME}:${IMAGE_TAG}
                    URL=http://${APP_NAME}-ci-${IMAGE_TAG}:3000

                    echo "Waiting for the container to start..."
                    for i in $(seq 1 20); do
                      curl -fs $URL/api/health > /dev/null && break
                      sleep 1
                    done

                    echo "--- health"; curl -fsS $URL/api/health; echo
                    echo "--- build info"; curl -fsS $URL/api/info | tee info.json; echo
                    grep -q "\\"build\\":\\"${BUILD_NUMBER}\\"" info.json

                    echo "--- create a task through the API"
                    curl -fsS -X POST -H "Content-Type: application/json" \
                         -d '{"title":"Pipeline test task","priority":"high","tag":"CI"}' $URL/api/tasks;
echo
                    curl -fsS $URL/api/tasks | grep -q "Pipeline test task"

                    echo "--- invalid input must be rejected"
                    CODE=$(curl -s -o /dev/null -w "%{http_code}" -X POST -H "Content-Type:
application/json" -d '{"title":""}' $URL/api/tasks)
                    test "$CODE" = "400"
                    echo "Container test passed"
                '''
```

```
            }
            post {
                always {
                    sh 'docker rm -f ${APP_NAME}-ci-${IMAGE_TAG} || true'
                }
            }
        }

        stage('Terraform Validate') {
            steps {
                dir('terraform') {
                    sh 'mkdir -p ${TF_PLUGIN_CACHE_DIR} ${TF_STATE_DIR}'
                    sh 'terraform version'
                    sh 'terraform fmt -check -recursive'
                    sh 'terraform init -no-color -backend-config="path=${TF_STATE_DIR}/default.tfstate" -
backend-config="workspace_dir=${TF_STATE_DIR}"'
                    sh 'terraform validate -no-color'
                }
            }
        }

        stage('Deploy to Staging') {
            steps {
                deploy('staging')
            }
        }

        stage('Smoke Test Staging') {
            steps {
                smokeTest('http://host.docker.internal:8081')
            }
        }

        stage('Approval') {
            when { expression { params.DEPLOY_TO_PRODUCTION } }
            steps {
                timeout(time: 15, unit: 'MINUTES') {
                    input message: "Staging is healthy. Deploy build #${env.BUILD_NUMBER} to production?",
ok: 'Deploy'
                }
            }
        }

        stage('Deploy to Production') {
            when { expression { params.DEPLOY_TO_PRODUCTION } }
            steps {
                deploy('production')
            }
        }

        stage('Smoke Test Production') {
            when { expression { params.DEPLOY_TO_PRODUCTION } }
            steps {
                smokeTest('http://host.docker.internal:8082')
            }
        }
    }

    post {
        success {
            echo "Build #${env.BUILD_NUMBER} (${env.GIT_SHORT}) deployed successfully."
            archiveArtifacts artifacts: 'terraform/*-plan.txt, test-results.xml', allowEmptyArchive: true
        }
        failure {
            echo "Build #${env.BUILD_NUMBER} failed. Check the stage logs above."
        }
        always {
            sh 'docker image rm ${APP_NAME}-test:${IMAGE_TAG} > /dev/null 2>&1 || true'
        }
    }
}

// Plans and applies one environment with Terraform.
def deploy(String envName) {
```

```
    dir('terraform') {
        sh "terraform workspace select -or-create ${envName}"
        sh """
            terraform plan -no-color -out=${envName}.tfplan \
              -var-file=environments/${envName}.tfvars \
              -var image_tag=${env.IMAGE_TAG} \
              -var app_version=${env.APP_VERSION} \
              -var build_number=${env.BUILD_NUMBER} \
              -var git_commit=${env.GIT_SHORT}
        """
        sh "terraform show -no-color ${envName}.tfplan > ${envName}-plan.txt"
        sh "terraform apply -no-color ${envName}.tfplan"
        sh 'terraform output'
    }
}

// Checks that the deployed environment serves this build, through the load balancer.
def smokeTest(String url) {
    sh """
        for i in \$(seq 1 30); do curl -fs ${url}/api/health > /dev/null && break; sleep 2; done
        echo "--- ${url}/api/info"
        curl -fsS ${url}/api/info | tee smoke.json; echo
        grep -q '"build":"${env.BUILD_NUMBER}"' smoke.json
        echo "--- requests through the load balancer"
        for i in 1 2 3 4 5 6; do curl -fsS ${url}/api/health; echo; done
        test "\$(curl -s -o /dev/null -w '%{http_code}' ${url}/)" = "200"
        echo "Smoke test passed for ${url}"
    """
}
```

Explanation of important parts

- **checkout scm** clones the exact commit that triggered the build. The short commit id and the version from package.json are stored in environment variables and shown as the build description.
- **Unit Tests** runs docker build --target test. The tests run inside the build, and the junit step publishes test-results.xml so Jenkins can show the results and the trend.
- **Build Image** tags the image with the build number and passes version, build number and commit as build arguments, so the running application knows which build it is.
- **Container Test** starts the new image on the Jenkins network and tests it through its API before it is deployed anywhere. The container is always removed in post { always }.
- **deploy()** selects (or creates) the Terraform workspace for the environment, saves the plan to a file, keeps a text copy of the plan as an artifact and applies exactly that plan.
- **input** pauses the build until someone approves the production deployment in Jenkins.
- **post** archives the plans and the test report when the build succeeds and removes the temporary test image every time.

8. TERRAFORM CONFIGURATION

8.1 Infrastructure as Code with Terraform

Terraform configuration is written in HCL. A provider connects Terraform to a platform (here the Docker engine), resources describe what must exist, variables make the code reusable, outputs return useful values, and the state file records what Terraform has created. The main commands are init (download providers), validate, plan (show the changes), apply (make the changes) and destroy.

8.2 Root Module

versions.tf pins the Terraform and provider versions. The backend is local, and its path is given at init time so that Jenkins can keep the state outside the build workspace.

File: terraform/versions.tf

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.6"
    }
  }

  # State is stored locally. The path is set with -backend-config
  # (Jenkins keeps it in /var/jenkins_home/tfstate so it survives workspace cleanup).
  backend "local" {}
}
```

File: terraform/providers.tf

```
provider "docker" {
  host = var.docker_host
}
```

variables.tf declares the inputs with types, descriptions, defaults and validation rules (for example environment must be dev, staging or production and replicas must be 1 to 5):

File: terraform/variables.tf

```
variable "environment" {
  description = "Deployment environment name"
  type        = string

  validation {
    condition     = contains(["dev", "staging", "production"], var.environment)
    error_message = "environment must be dev, staging or production."
  }
}

variable "image_tag" {
  description = "Tag of the taskflow Docker image to deploy (Jenkins passes the build number)"
  type        = string
  default     = "latest"
}

variable "replicas" {
  description = "Number of application containers behind the load balancer"
  type        = number
  default     = 2

  validation {
    condition     = var.replicas >= 1 && var.replicas <= 5
    error_message = "replicas must be between 1 and 5."
  }
}

variable "external_port" {
  description = "Port on the host where the load balancer is published"
  type        = number
}

variable "app_version" {
  description = "Application version shown in the UI"
  type        = string
  default     = "1.0.0"
}

variable "build_number" {
  description = "Jenkins build number that produced the image"
  type        = string
  default     = "manual"
}

variable "git_commit" {
  description = "Git commit of the deployed code"
  type        = string
```

```
  default     = "unknown"
}

variable "memory_mb" {
  description = "Memory limit for each application container (MB)"
  type        = number
  default     = 128
}

variable "docker_host" {
  description = "Docker engine endpoint (Linux socket by default)"
  type        = string
  default     = "unix:///var/run/docker.sock"
}
```

main.tf uses the reusable module and passes the image tag, replica count, port, memory limit, labels and the environment variables that the application shows in its side panel:

File: terraform/main.tf

```
# Root module: deploys one TaskFlow environment using the reusable webapp module.

locals {
  # Labels on the network and volume must stay the same between deployments,
  # because changing a label forces Docker to recreate the resource (and the
  # volume holds the task data). The build number is only put on containers.
  common_labels = {
    project     = "taskflow"
    environment = var.environment
    managed_by  = "terraform"
  }
}

module "taskflow" {
  source = "./modules/webapp"

  name          = "taskflow-${var.environment}"
  image         = "taskflow:${var.image_tag}"
  replicas      = var.replicas
  external_port = var.external_port
  memory_mb     = var.memory_mb
  labels        = local.common_labels

  container_labels = {
    build = var.build_number
  }

  env = {
    DEPLOY_ENV   = var.environment
    APP_VERSION  = var.app_version
    BUILD_NUMBER = var.build_number
    GIT_COMMIT   = var.git_commit
  }
}
```

File: terraform/outputs.tf

```
output "app_url" {
  description = "URL of the load balancer for this environment"
  value       = module.taskflow.url
}

output "app_containers" {
  description = "Names of the application containers"
  value       = module.taskflow.app_containers
}

output "load_balancer" {
  description = "Name of the load balancer container"
  value       = module.taskflow.load_balancer
}

output "network" {
  description = "Docker network created for this environment"
```

```
  value       = module.taskflow.network
}

output "data_volume" {
  description = "Docker volume that stores the tasks"
  value       = module.taskflow.data_volume
}

output "deployed_image" {
  description = "Image deployed to this environment"
  value       = module.taskflow.image
}
```

Each environment has its own variable file:

File: terraform/environments/staging.tfvars

```
environment   = "staging"
replicas      = 2
external_port = 8081
memory_mb     = 128
```

File: terraform/environments/production.tfvars

```
environment   = "production"
replicas      = 3
external_port = 8082
memory_mb     = 192
```

File: terraform/environments/dev.tfvars

```
environment   = "dev"
replicas      = 1
external_port = 8083
memory_mb     = 128
```

8.3 Reusable Module: webapp

The module creates everything one environment needs. Because the name, image, replica count, port and labels are inputs, the same module is used for dev, staging and production, and it could be reused for another web application.

File: terraform/modules/webapp/main.tf

```
# Reusable module: a web application running as N containers behind an Nginx
# load balancer, with its own network and a persistent data volume.

resource "docker_network" "this" {
  name   = "${var.name}-net"
  driver = "bridge"

  dynamic "labels" {
    for_each = var.labels
    content {
      label = labels.key
      value = labels.value
    }
  }
}

resource "docker_volume" "data" {
  name = "${var.name}-data"

  dynamic "labels" {
    for_each = var.labels
    content {
      label = labels.key
      value = labels.value
    }
  }
}

# The application image is built by Jenkins, so it is read from the local engine.
data "docker_image" "app" {
  name = var.image
```

```
}

resource "docker_image" "nginx" {
  name         = var.lb_image
  keep_locally = true
}

resource "docker_container" "app" {
  count = var.replicas

  name    = "${var.name}-app-${count.index + 1}"
  image   = data.docker_image.app.id
  restart = "unless-stopped"
  memory  = var.memory_mb

  env = [for k, v in merge(var.env, { INSTANCE_NO = tostring(count.index + 1) }) : "${k}=${v}"]

  networks_advanced {
    name = docker_network.this.name
  }

  volumes {
    volume_name    = docker_volume.data.name
    container_path = "/data"
  }

  healthcheck {
    test     = ["CMD", "wget", "-qO-", "http://127.0.0.1:${var.app_port}/api/health"]
    interval = "10s"
    timeout  = "3s"
    retries  = 3
  }

  # do not report success until the container is healthy
  wait         = true
  wait_timeout = 90

  dynamic "labels" {
    for_each = merge(var.labels, var.container_labels, { role = "app" })
    content {
      label = labels.key
      value = labels.value
    }
  }
}

resource "docker_container" "lb" {
  name    = "${var.name}-lb"
  image   = docker_image.nginx.image_id
  restart = "unless-stopped"

  ports {
    internal = 80
    external = var.external_port
  }

  networks_advanced {
    name = docker_network.this.name
  }

  # Nginx config is generated from the list of app containers
  upload {
    file = "/etc/nginx/conf.d/default.conf"
    content = templatefile("${path.module}/templates/nginx.conf.tftpl", {
      name      = var.name
      upstreams = docker_container.app[*].name
      app_port  = var.app_port
    })
  }

  dynamic "labels" {
    for_each = merge(var.labels, var.container_labels, { role = "load-balancer" })
    content {
      label = labels.key
```

```
      value = labels.value
    }
  }
}
```

Explanation

- **docker_network** and **docker_volume** give each environment its own network and data storage.
- **data "docker_image"** reads the image built by Jenkins from the local engine, so it is never pulled from a registry.
- **docker_container "app"** uses count to create the required number of replicas. Each container has a memory limit, a health check, the data volume and a restart policy. wait = true makes Terraform wait until the container is healthy.
- **docker_container "lb"** runs Nginx. Its configuration is generated with templatefile() from the list of app container names, so it always matches the replica count.
- **dynamic "labels"** adds the labels from a map to every resource.

File: terraform/modules/webapp/templates/nginx.conf.tftpl

```
# Generated by Terraform for ${name}
upstream app_backend {
    zone app_backend 64k;
%{ for host in upstreams ~}
    server ${host}:${app_port} max_fails=3 fail_timeout=10s;
%{ endfor ~}
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://app_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_next_upstream error timeout http_502 http_503;
    }

    location = /lb-health {
        default_type text/plain;
        return 200 "ok\n";
    }
}
```

File: terraform/modules/webapp/variables.tf

```
variable "name" {
  description = "Prefix used for every resource name"
  type        = string
}

variable "image" {
  description = "Application image (must exist in the local Docker engine)"
  type        = string
}

variable "replicas" {
  description = "Number of application containers"
  type        = number
  default     = 1
}

variable "app_port" {
  description = "Port the application listens on inside the container"
  type        = number
  default     = 3000
}

variable "external_port" {
```

```
  description = "Host port for the load balancer"
  type        = number
}

variable "memory_mb" {
  description = "Memory limit per application container (MB)"
  type        = number
  default     = 128
}

variable "env" {
  description = "Environment variables passed to the application"
  type        = map(string)
  default     = {}
}

variable "labels" {
  description = "Labels added to every resource"
  type        = map(string)
  default     = {}
}

variable "container_labels" {
  description = "Extra labels for containers only (values may change on every deploy)"
  type        = map(string)
  default     = {}
}

variable "lb_image" {
  description = "Image used for the load balancer"
  type        = string
  default     = "nginx:1.27-alpine"
}
```

8.4 State and Workspaces

Staging and production use the same code but must have separate state. Terraform workspaces are used for this: the pipeline runs terraform workspace select -or-create staging (or production) before planning. In Jenkins the state files are kept in /var/jenkins_home/tfstate/taskflow so they are not lost when the workspace is cleaned. The dependency lock file (.terraform.lock.hcl) is committed with hashes for Windows and Linux, so the same provider version is used on the laptop and in Jenkins.

9. IMPLEMENTATION AND EXECUTION

9.1 Terraform Workflow (dev environment)

Before using Terraform in the pipeline, the configuration was run by hand from PowerShell for the dev environment. On Windows the Docker engine is reached through a named pipe, so docker_host is set on the command line.

terraform version

terraform init

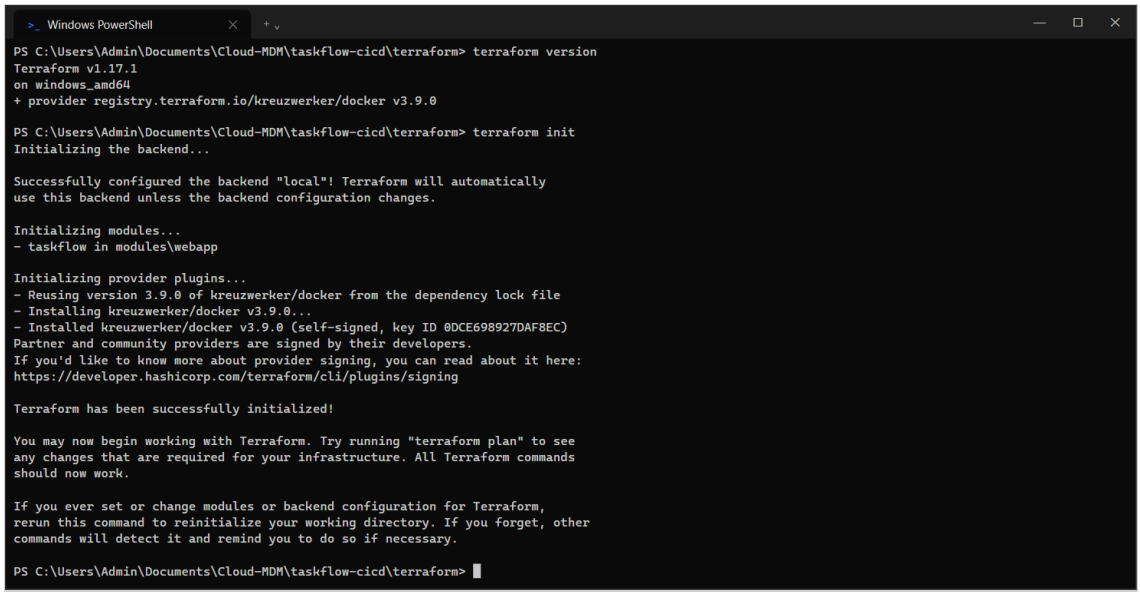


Figure 7: terraform init downloads the kreuzwerker/docker provider using the lock file

terraform fmt -check -recursive

terraform validate



Figure 8: Formatting check and validation passed

terraform plan -var-file="environments/dev.tfvars" -var docker_host="npipe:////./pipe/docker_engine" -out dev.tfplan

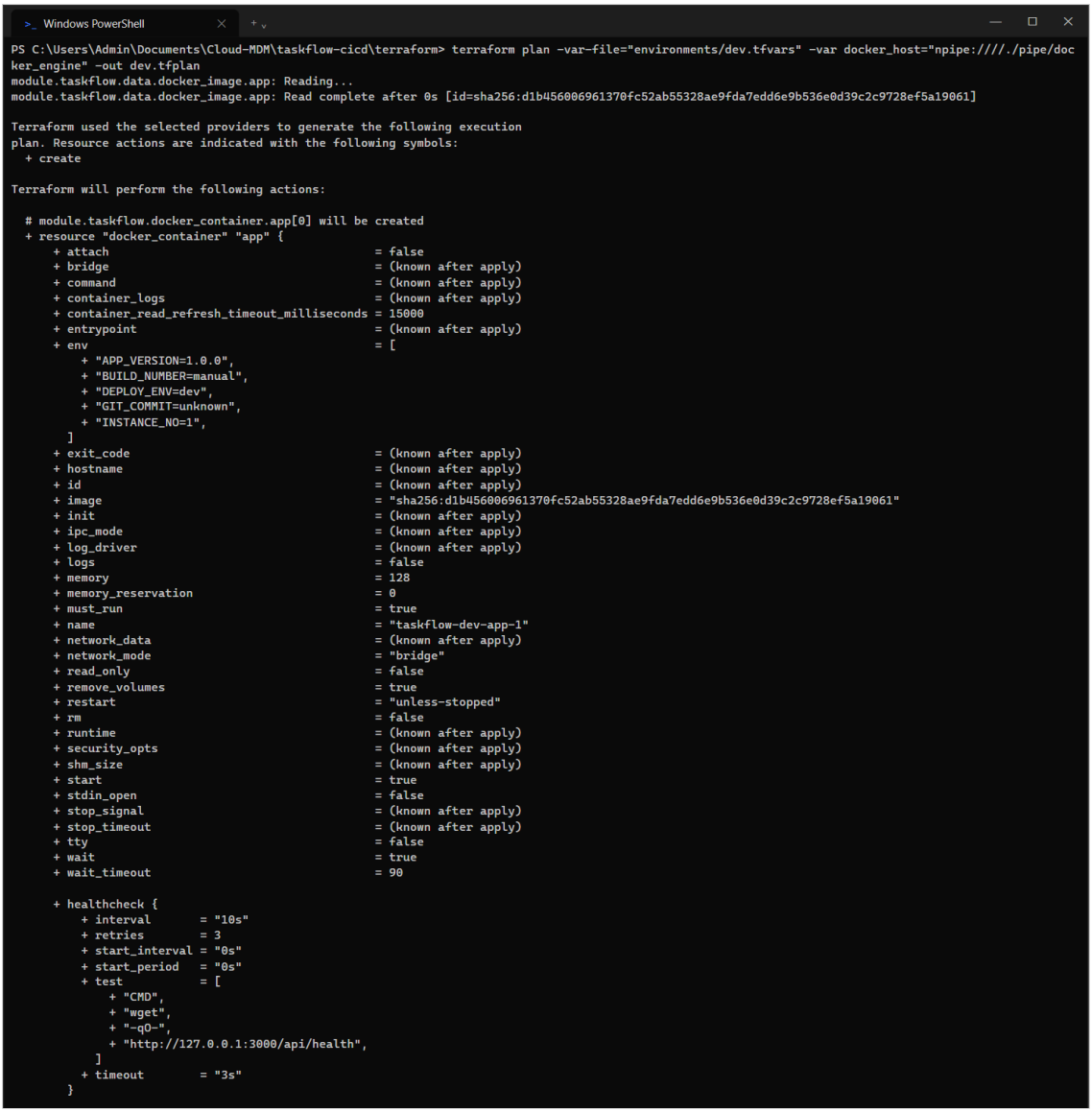


Figure 9: Terraform plan (start): the app container with environment variables, memory limit and health check

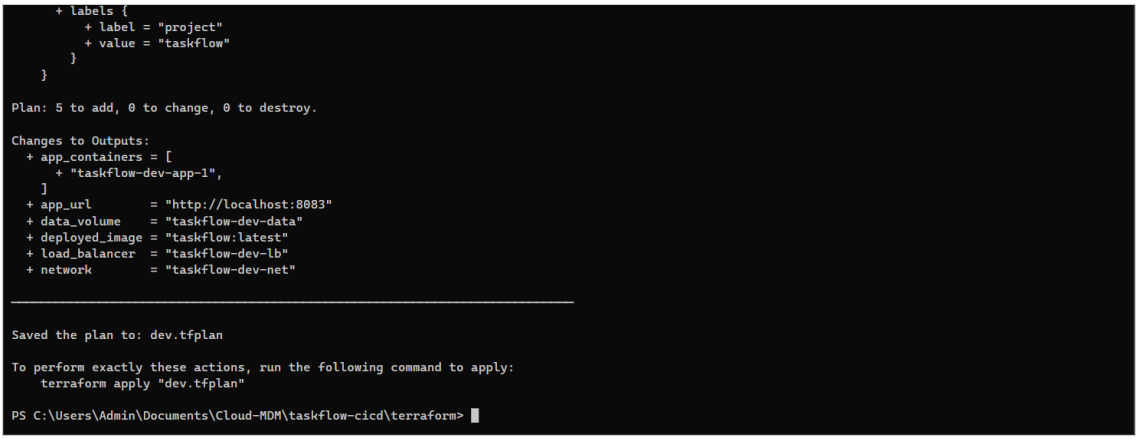


Figure 10: Terraform plan (end): 5 resources to add and the outputs

terraform apply dev.tfplan

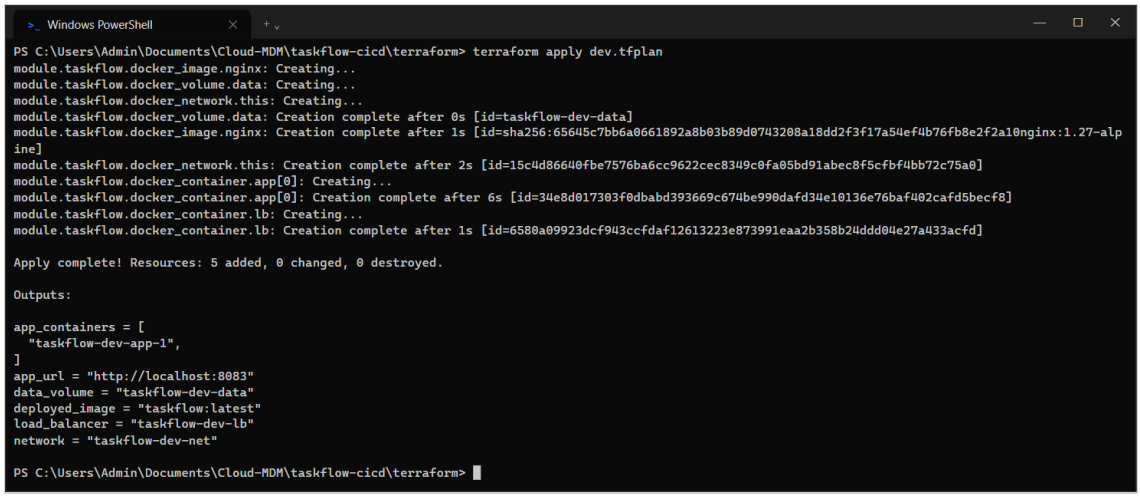


Figure 11: terraform apply: network, volume, image, app container and load balancer created

terraform output

terraform state list

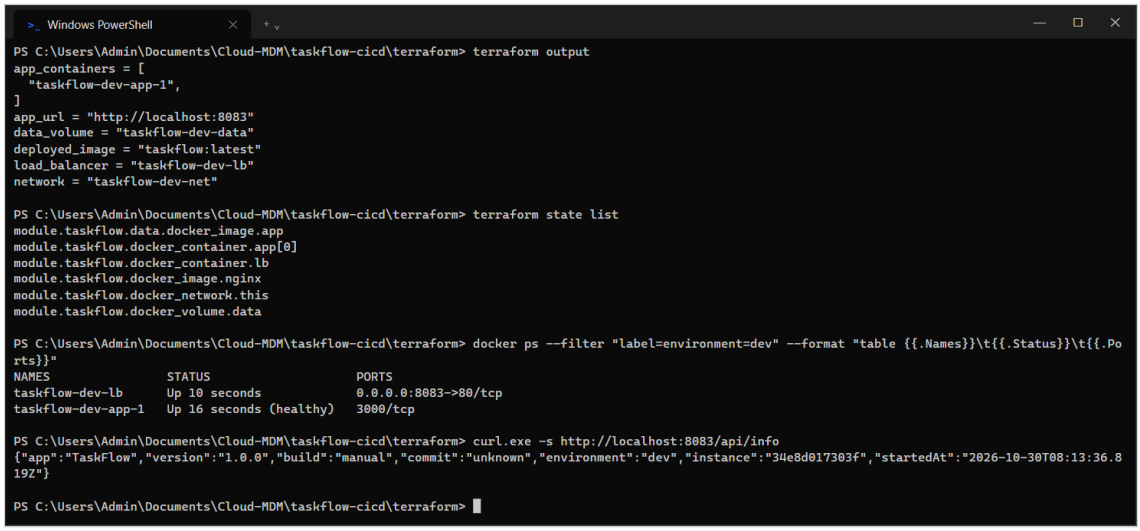


Figure 12: Outputs, resources in the state and the running dev containers

To show that the configuration is reusable, the number of replicas was changed with a single variable:

terraform apply -var-file="environments/dev.tfvars" -var replicas=2 -auto-approve

```
    }

Plan: 3 to add, 0 to change, 2 to destroy.

Changes to Outputs:
  ~ app_containers = [
      - "taskflow-dev-app-1",
      + "taskflow-dev-app-2",
    ]
module.taskflow.docker_container.lb: Destroying... [id=6580a09923dcf943ccfdaf12613223e873991eaa2b358b24ddd04e27a433acfd]
module.taskflow.docker_container.lb: Destruction complete after 1s
module.taskflow.docker_container.app[0]: Destroying... [id=34e8d017303f0dbabd393669c674be990dafd34e10136e76baf402cafd5becf8]
module.taskflow.docker_container.app[1]: Creating...
module.taskflow.docker_container.app[0]: Destruction complete after 0s
module.taskflow.docker_container.app[0]: Creating...
module.taskflow.docker_container.app[1]: Creation complete after 6s [id=a0ce9e7cf7a12e8553edffe5ad10617fde3e6d4032748730f475ada9956bd9a5]
module.taskflow.docker_container.app[0]: Creation complete after 6s [id=c7422a7b9dc08eb5f24fbae811e6161a3484b2f2f5aae2ab8c97deb6bf3d5e1a]
module.taskflow.docker_container.lb: Creating...
module.taskflow.docker_container.lb: Creation complete after 1s [id=437028beeda24e68e38d0aa4152841637021c1810be7f05524688886332c89861]

Apply complete! Resources: 3 added, 0 changed, 2 destroyed.

Outputs:

app_containers = [
  "taskflow-dev-app-1",
  "taskflow-dev-app-2",
]
app_url = "http://localhost:8083"
data_volume = "taskflow-dev-data"
deployed_image = "taskflow:latest"
load_balancer = "taskflow-dev-lb"
network = "taskflow-dev-net"

PS C:\Users\Admin\Documents\Cloud-MDM\taskflow-cicd\terraform> docker ps --filter "label=environment=dev" --format "table {{.Names}}\t{{.Status}}\t{{.Po
rts}}"
NAMES               STATUS                    PORTS
taskflow-dev-lb     Up Less than a second     0.0.0.0:8083->80/tcp
taskflow-dev-app-1  Up 6 seconds (healthy)   3000/tcp
taskflow-dev-app-2  Up 6 seconds (healthy)   3000/tcp

PS C:\Users\Admin\Documents\Cloud-MDM\taskflow-cicd\terraform>
```

Figure 13: Scaling dev to 2 replicas: a second app container is added and the load balancer is recreated with the new backend list

Finally the dev environment was removed:

terraform destroy -var-file="environments/dev.tfvars" -auto-approve

```
    }

Plan: 0 to add, 0 to change, 6 to destroy.

Changes to Outputs:
  - app_containers = [
      - "taskflow-dev-app-1",
    ] -> null
  - app_url        = "http://localhost:8083" -> null
  - data_volume    = "taskflow-dev-data" -> null
  - deployed_image = "taskflow:latest" -> null
  - load_balancer  = "taskflow-dev-lb" -> null
  - network        = "taskflow-dev-net" -> null
module.taskflow.docker_container.lb: Destroying... [id=437028beeda24e68e38d0aa4152841637021c1810be7f05524688886332c89861]
module.taskflow.docker_container.lb: Destruction complete after 0s
module.taskflow.docker_image.nginx: Destroying... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
module.taskflow.docker_image.nginx: Destruction complete after 0s
module.taskflow.docker_container.app[1]: Destroying... [id=a0ce9e7cf7a12e8553edffe5ad10617fde3e6d4032748730f475ada9956bd9a5]
module.taskflow.docker_container.app[0]: Destroying... [id=c7422a7b9dc08eb5f24fbae811e6161a3484b2f2f5aae2ab8c97deb6bf3d5e1a]
module.taskflow.docker_container.app[1]: Destruction complete after 0s
module.taskflow.docker_container.app[0]: Destruction complete after 0s
module.taskflow.docker_network.this: Destroying... [id=15c4d86640fbe7576ba6cc9622cec8349c0fa05bd91abec8f5cfbf4bb72c75a0]
module.taskflow.docker_volume.data: Destroying... [id=taskflow-dev-data]
module.taskflow.docker_volume.data: Destruction complete after 2s
module.taskflow.docker_network.this: Destruction complete after 3s

Destroy complete! Resources: 6 destroyed.

PS C:\Users\Admin\Documents\Cloud-MDM\taskflow-cicd\terraform> docker ps --filter "label=environment=dev"
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

PS C:\Users\Admin\Documents\Cloud-MDM\taskflow-cicd\terraform>
```

Figure 14: terraform destroy removed all 6 resources

9.2 Pipeline Triggered by a Commit

The first manual build of the job (build #1) failed during checkout because the Jenkins container could not resolve github.com for a short time. After the network recovered, build #2 completed all stages. From then on, builds were started by Jenkins itself. To show this, version 1.1.0 was committed and pushed to GitHub:

git commit -m "Release 1.1.0: checkout code only once in the pipeline"

git push

Within two minutes Poll SCM found the new commit and started build #3. The build page shows **Started by an SCM change**, the Git revision and the commit message:

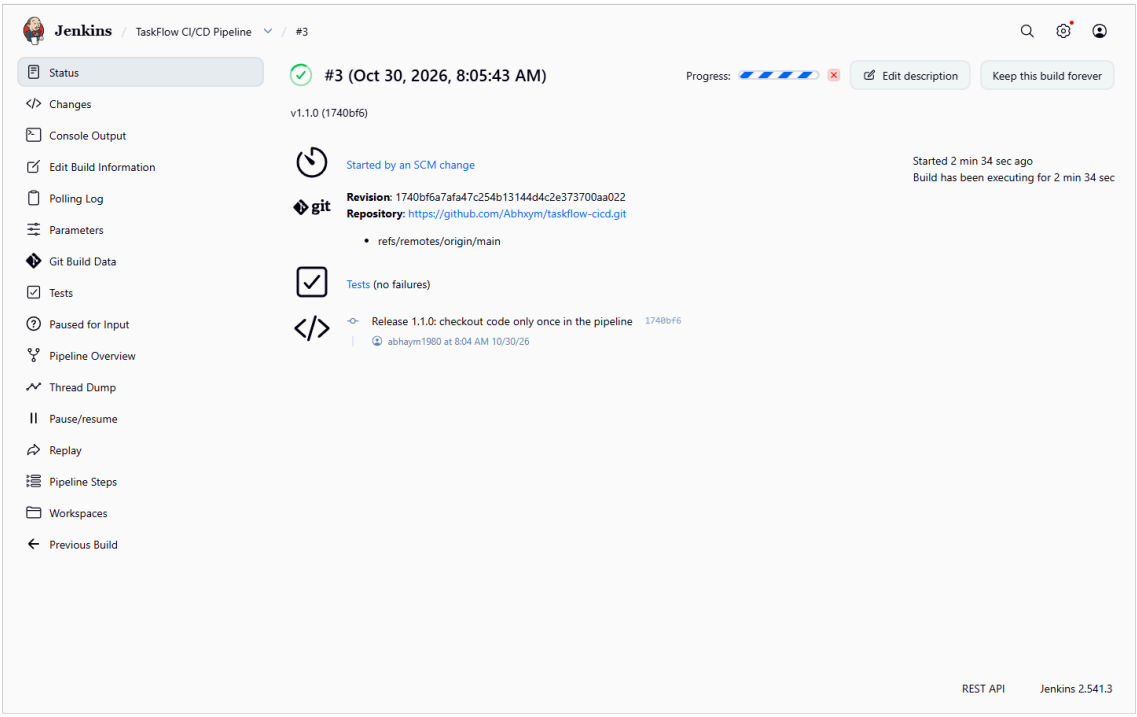


Figure 15: Build #3 started automatically by the SCM change

9.3 Pipeline Stages in the Console Output

Checkout

```
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] checkout
08:05:49  Selected Git installation does not exist. Using Default
08:05:49  The recommended git tool is: NONE
08:05:49  No credentials specified
08:05:49   > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/taskflow-pipeline/.git # timeout=10
08:05:49  Fetching changes from the remote Git repository
08:05:49   > git config remote.origin.url https://github.com/Abhxym/taskflow-cicd.git # timeout=10
08:05:49  Fetching upstream changes from https://github.com/Abhxym/taskflow-cicd.git
08:05:49   > git --version # timeout=10
08:05:49   > git --version # 'git version 2.47.3'
08:05:49   > git fetch --tags --force --progress -- https://github.com/Abhxym/taskflow-cicd.git +refs/heads/*:refs/remotes/origin/*
# timeout=10
08:05:51   > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
08:05:51  Checking out Revision 1740bf6a7afa47c254b13144d4c2e373700aa022 (refs/remotes/origin/main)
08:05:51   > git config core.sparsecheckout # timeout=10
08:05:51   > git checkout -f 1740bf6a7afa47c254b13144d4c2e373700aa022 # timeout=10
08:05:51  Commit message: "Release 1.1.0: checkout code only once in the pipeline"
08:05:51   > git rev-list --no-walk a88155c70dc43adf5bc6ee203656d07dbec6cb23 # timeout=10
[Pipeline] script
[Pipeline] {
```

Figure 16: Checkout stage: clone from GitHub, version 1.1.0 and commit 1740bf6

Unit Tests

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Unit Tests)
[Pipeline] sh
08:05:54  + docker build --target test -t taskflow-test:3 .
08:05:54  DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
08:05:54              Install the buildx component to build images with BuildKit:
08:05:54              https://docs.docker.com/go/buildx/
08:05:54
08:05:54  Sending build context to Docker daemon   51.2kB
08:05:54
08:05:54  Step 1/7 : FROM node:22-alpine AS test
08:05:54   ---> fe4f6d74e203
08:05:54  Step 2/7 : WORKDIR /src
08:05:54   ---> Running in 5333e4bbe8b5
08:05:56   ---> Removed intermediate container 5333e4bbe8b5
08:05:56   ---> f0f089bedd57
08:05:56  Step 3/7 : COPY package.json ./
08:05:57   ---> 8324d3b4fa8b
08:05:57  Step 4/7 : COPY app ./app
08:05:58   ---> b9d3ed2c5be6
08:05:58  Step 5/7 : COPY public ./public
08:06:00   ---> 4b7a2a396818
08:06:00  Step 6/7 : COPY test ./test
08:06:04   ---> 1e297ecf1e8d
08:06:04  Step 7/7 : RUN node --test       --test-reporter=spec --test-reporter-destination=stdout       --test-reporter=junit --
test-reporter-destination=test-results.xml
08:06:04   ---> Running in 8846d663d25e
08:06:05  ✓ health endpoint returns ok (101.144668ms)
08:06:05  ✓ info endpoint returns build details (7.151635ms)
08:06:05  ✓ seed tasks are created on first start (5.916504ms)
08:06:05  ✓ create, update and delete a task (46.09293ms)
08:06:05  ✓ serves the web page (8.253959ms)
```

Figure 17: Unit Tests stage: tests run inside the Docker build

Build Image

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Image)
[Pipeline] sh
08:06:09  + docker build --build-arg APP_VERSION=1.1.0 --build-arg BUILD_NUMBER=3 --build-arg GIT_COMMIT=1740bf6 -t taskflow:3 -t
taskflow:latest .
08:06:09  DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
08:06:09              Install the buildx component to build images with BuildKit:
08:06:09              https://docs.docker.com/go/buildx/
08:06:09
08:06:09  Sending build context to Docker daemon   51.2kB
08:06:09
08:06:09  Step 1/23 : FROM node:22-alpine AS test
08:06:09   ---> fe4f6d74e203
08:06:09  Step 2/23 : WORKDIR /src
08:06:09   ---> Using cache
08:06:09   ---> f0f089bedd57
08:06:09  Step 3/23 : COPY package.json ./
08:06:09   ---> Using cache
08:06:09   ---> 8324d3b4fa8b
08:06:09  Step 4/23 : COPY app ./app
08:06:09   ---> Using cache
08:06:09   ---> b9d3ed2c5be6
08:06:09  Step 5/23 : COPY public ./public
08:06:09   ---> Using cache
08:06:09   ---> 4b7a2a396818
08:06:09  Step 6/23 : COPY test ./test
```

Figure 18: Build Image stage: image tagged taskflow:3 with build arguments

Container Test

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Container Test)
[Pipeline] sh
08:06:28  + docker run -d --name taskflow-ci-3 --network ci-net taskflow:3
08:06:28  b7f740acbb168389371563b9d13b20ae525a6e1b5837604a5df425f9c2302e8c
08:06:28  + URL=http://taskflow-ci-3:3000
08:06:28  + echo Waiting for the container to start...
08:06:28  Waiting for the container to start...
08:06:28  + seq 1 20
08:06:28  + curl -fs http://taskflow-ci-3:3000/api/health
08:06:28  + sleep 1
08:06:30  + curl -fs http://taskflow-ci-3:3000/api/health
08:06:30  + break
08:06:30  + echo --- health
08:06:30  --- health
08:06:30  + curl -fsS http://taskflow-ci-3:3000/api/health
08:06:30  {"status":"ok","instance":"b7f740acbb16"}+ echo
08:06:30
08:06:30  + echo --- build info
08:06:30  --- build info
08:06:30  + curl -fsS http://taskflow-ci-3:3000/api/info
08:06:30  + tee info.json
08:06:30
{"app":"TaskFlow","version":"1.1.0","build":"3","commit":"1740bf6","environment":"local","instance":"b7f740acbb16","startedAt":"202
6-10-30T08:06:28.831Z"}+ echo
```

Figure 19: Container Test stage: the new image answers with build 3 and commit 1740bf6

Terraform Validate

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Terraform Validate)
[Pipeline] dir
08:06:31  Running in /var/jenkins_home/workspace/taskflow-pipeline/terraform
[Pipeline] {
[Pipeline] sh
08:06:31  + mkdir -p /var/jenkins_home/.terraform.d/plugin-cache /var/jenkins_home/tfstate/taskflow
[Pipeline] sh
08:06:31  + terraform version
08:06:31  Terraform v1.17.1
08:06:31  on linux_amd64
08:06:31  + provider registry.terraform.io/kreuzwerker/docker v3.9.0
08:06:31
[Pipeline] sh
08:06:32  + terraform fmt -check -recursive
[Pipeline] sh
08:06:32  + terraform init -no-color -backend-config=path=/var/jenkins_home/tfstate/taskflow/default.tfstate -backend-
config=workspace_dir=/var/jenkins_home/tfstate/taskflow
08:06:32  Initializing the backend...
08:06:32
08:06:32  Initializing modules...
08:06:32
08:06:32  Initializing provider plugins...
08:06:32  - Reusing version 3.9.0 of kreuzwerker/docker from the dependency lock file
08:06:34  - Using previously-installed kreuzwerker/docker v3.9.0
08:06:34
08:06:34  Terraform has been successfully initialized!
```

Figure 20: Terraform Validate stage

Deploy to Staging

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Staging)
[Pipeline] dir
08:06:35  Running in /var/jenkins_home/workspace/taskflow-pipeline/terraform
[Pipeline] {
[Pipeline] sh
08:06:35  + terraform workspace select -or-create staging
08:06:35  Switched to workspace "staging".
[Pipeline] sh
08:06:35  + terraform plan -no-color -out=staging.tfplan -var-file=environments/staging.tfvars -var image_tag=3 -var
app_version=1.1.0 -var build_number=3 -var git_commit=1740bf6
08:06:35  module.taskflow.data.docker_image.app: Reading...
08:06:35  module.taskflow.docker_image.nginx: Refreshing state...
[id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
08:06:35  module.taskflow.docker_volume.data: Refreshing state... [id=taskflow-staging-data]
08:06:35  module.taskflow.docker_network.this: Refreshing state...
[id=b9e4128e2294341d7e71429b65da62ef41297a58b9306f9a0e09b08a2914cb55]
08:06:36  module.taskflow.data.docker_image.app: Read complete after 0s
[id=sha256:d1b456006961370fc52ab55328ae9fda7edd6e9b536e0d39c2c9728ef5a19061]
08:06:37  module.taskflow.docker_container.app[1]: Refreshing state...
[id=f11f7d8c31c55fd6e547261b420a5fa3845ad04c966bfb5d7f66ee85db81866d]
08:06:37  module.taskflow.docker_container.app[0]: Refreshing state...
[id=b4bcda9f6ccabeab0e1c33a544be4fe2440d069cdf466f41a03ee51ee036aec3]
08:06:38  module.taskflow.docker_container.lb: Refreshing state...
[id=e37c79930d9f3f002c1b8c68038a989a40161f49ded5d8daab7415217bef9552]
08:06:38
08:06:38  Terraform used the selected providers to generate the following execution
08:06:38  plan. Resource actions are indicated with the following symbols:
08:06:38  -/+ destroy and then create replacement
08:06:38
08:06:38  Terraform will perform the following actions:
08:06:38
08:06:38    # module.taskflow.docker_container.app[0] must be replaced
08:06:38  -/+ resource "docker_container" "app" {
08:06:38        + bridge                                    = (known after apply)
08:06:38        ~ command                                   = [
08:06:38            - "node",
08:06:38            - "app/server.js",
08:06:38          ] -> (known after apply)
08:06:38        + container_logs                            = (known after apply)
08:06:38        - cpu_shares                                = 0 -> null
08:06:38        - dns                                       = [] -> null
```

Figure 21: Deploy to Staging: Terraform workspace staging and plan

```
08:06:38            + label = "build"
08:06:38            + value = "3"
08:06:38          }
08:06:38
08:06:38          # (3 unchanged blocks hidden)
08:06:38      }
08:06:38
08:06:38  Plan: 5 to add, 0 to change, 5 to destroy.
08:06:38
08:06:38  Changes to Outputs:
08:06:38    ~ deployed_image = "taskflow:2" -> "taskflow:3"
[Pipeline] sh
08:06:38  + terraform show -no-color staging.tfplan
[Pipeline] sh
08:06:39  + terraform apply -no-color staging.tfplan
08:06:39  module.taskflow.docker_container.lb: Destroying... [id=e37c79930d9f3f002c1b8c68038a989a40161f49ded5d8daab7415217bef9552]
08:06:39  module.taskflow.docker_container.lb: Destruction complete after 1s
08:06:39  module.taskflow.docker_container.app[0]: Destroying...
[id=b4bcda9f6ccabeab0e1c33a544be4fe2440d069cdf466f41a03ee51ee036aec3]
08:06:39  module.taskflow.docker_container.app[1]: Destroying...
[id=f11f7d8c31c55fd6e547261b420a5fa3845ad04c966bfb5d7f66ee85db81866d]
08:06:40  module.taskflow.docker_container.app[1]: Destruction complete after 0s
08:06:40  module.taskflow.docker_container.app[0]: Destruction complete after 0s
08:06:40  module.taskflow.docker_volume.data: Destroying... [id=taskflow-staging-data]
08:06:40  module.taskflow.docker_network.this: Destroying... [id=b9e4128e2294341d7e71429b65da62ef41297a58b9306f9a0e09b08a2914cb55]
08:06:42  module.taskflow.docker_volume.data: Destruction complete after 2s
08:06:42  module.taskflow.docker_volume.data: Creating...
08:06:42  module.taskflow.docker_volume.data: Creation complete after 0s [id=taskflow-staging-data]
08:06:42  module.taskflow.docker_network.this: Destruction complete after 2s
08:06:42  module.taskflow.docker_network.this: Creating...
08:06:45  module.taskflow.docker_network.this: Creation complete after 2s
[id=ffbe1f8deae006b11bd91c9bc4a4f92eea94368c437fae5aabc3c102fd771dd2]
08:06:45  module.taskflow.docker_container.app[1]: Creating...
```

Figure 22: Terraform apply in staging completed

Smoke Test Staging

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Smoke Test Staging)
[Pipeline] sh
08:06:56  + seq 1 30
08:06:56  + curl -fs http://host.docker.internal:8081/api/health
08:06:56  + break
08:06:56  + echo --- http://host.docker.internal:8081/api/info
08:06:56  --- http://host.docker.internal:8081/api/info
08:06:56  + curl -fsS http://host.docker.internal:8081/api/info
08:06:56  + tee smoke.json
08:06:56
{"app":"TaskFlow","version":"1.1.0","build":"3","commit":"1740bf6","environment":"staging","instance":"52e114eef47f","startedAt":"2
026-10-30T08:06:45.569Z"}+ echo
08:06:56
08:06:56  + grep -q "build":"3" smoke.json
08:06:56  + echo --- requests through the load balancer
08:06:56  --- requests through the load balancer
08:06:56  + curl -fsS http://host.docker.internal:8081/api/health
```

Figure 23: Smoke test through the staging load balancer

9.4 Manual Approval

After the staging smoke test, the pipeline paused. The Stage View shows all stages up to Approval as successful and the Approval stage waiting:

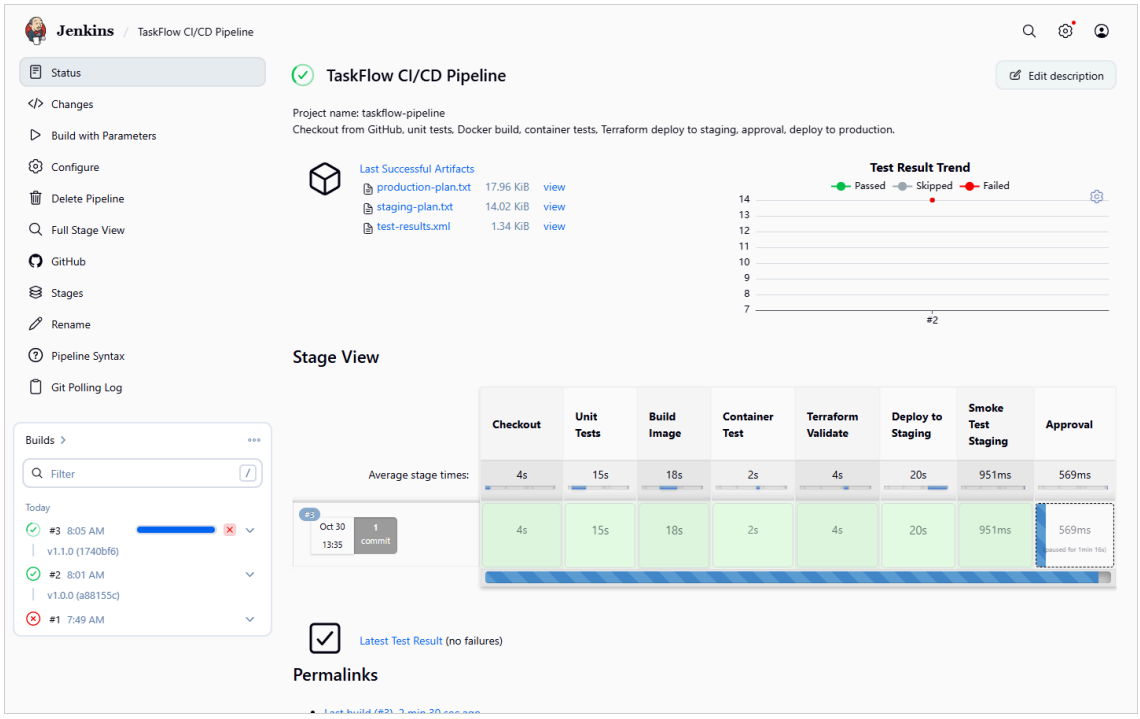


Figure 24: Stage View: build #3 waiting at the Approval stage

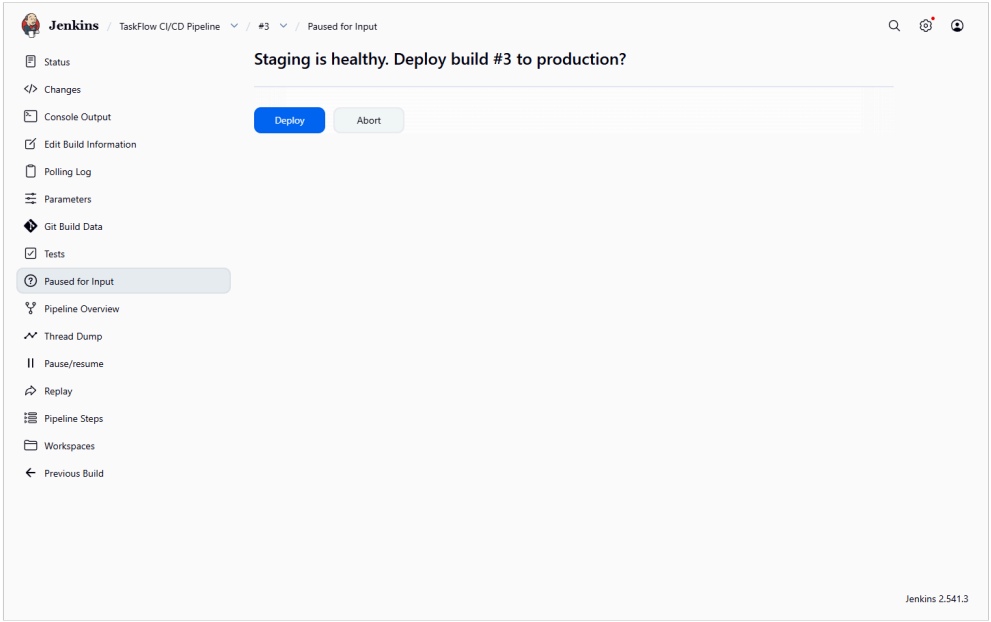


Figure 25: Approval page in Jenkins: Deploy or Abort

At this point staging already ran version 1.1.0 (build 3), while production still ran version 1.0.0 (build 2):

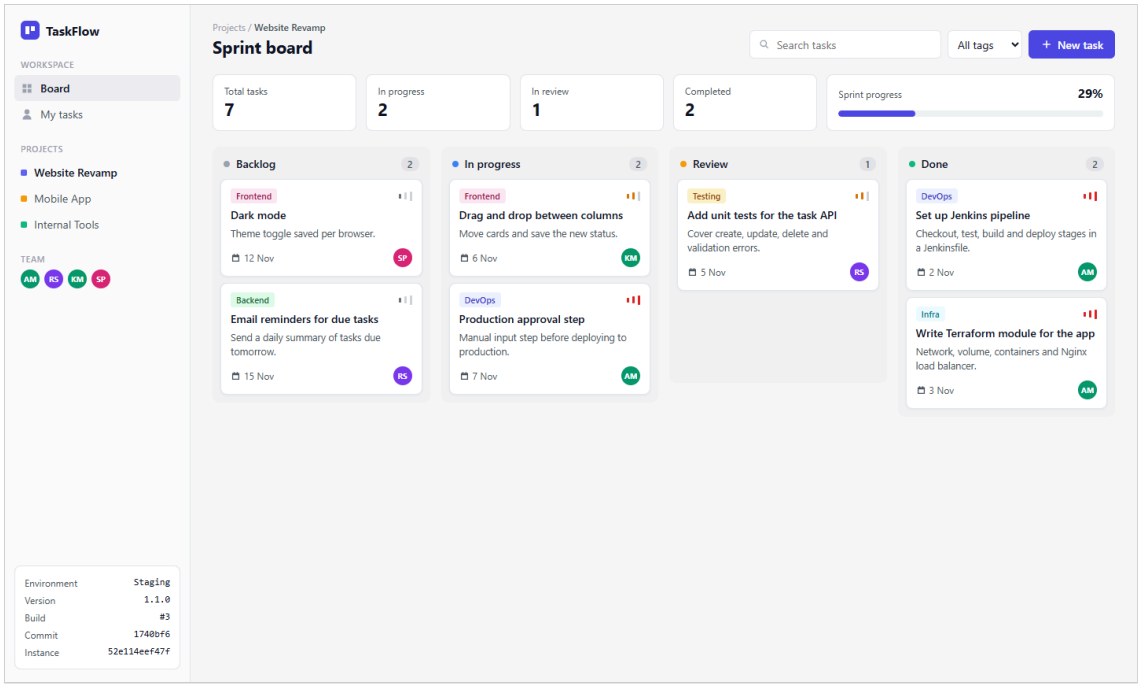


Figure 26: Staging: version 1.1.0, build #3, commit 1740bf6, 2 instances

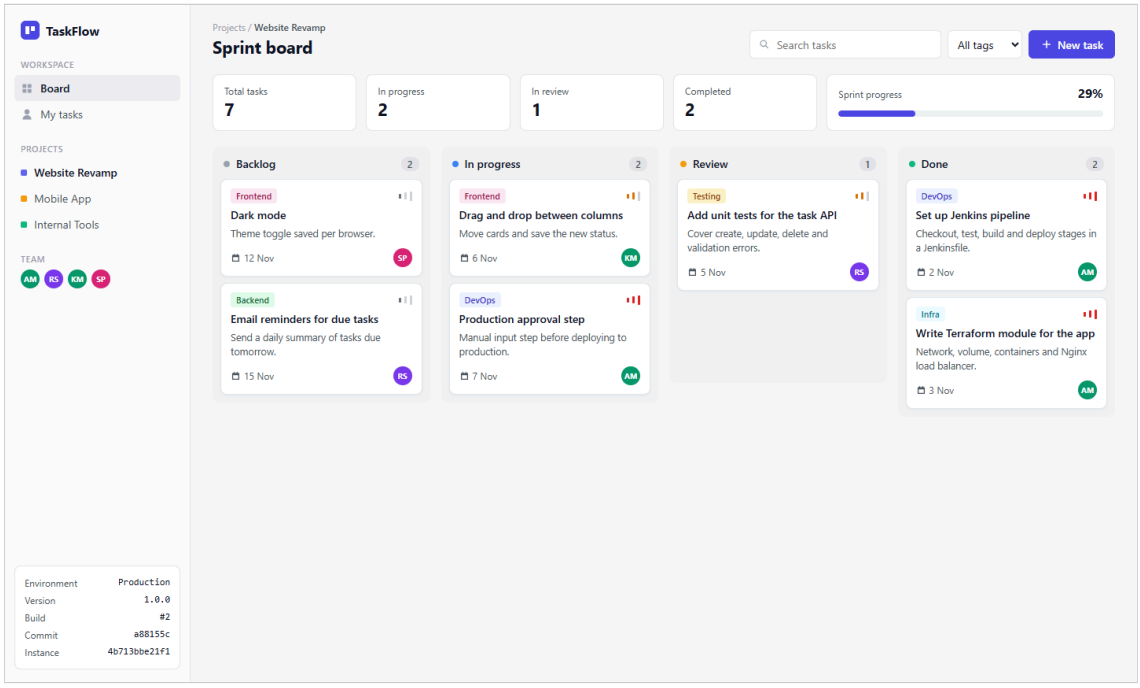


Figure 27: Production before approval: still version 1.0.0, build #2

After clicking Deploy, the pipeline deployed the same image to production and ran the final smoke test:

```
08:08:47            + label = "build"
08:08:47            + value = "3"
08:08:47          }
08:08:47
08:08:47          # (3 unchanged blocks hidden)
08:08:47      }
08:08:47
08:08:47  Plan: 6 to add, 0 to change, 6 to destroy.
08:08:47
08:08:47  Changes to Outputs:
08:08:47    ~ deployed_image = "taskflow:2" -> "taskflow:3"
[Pipeline] sh
08:08:47  + terraform show -no-color production.tfplan
[Pipeline] sh
08:08:47  + terraform apply -no-color production.tfplan
08:08:47  module.taskflow.docker_container.lb: Destroying... [id=b2c69c9b2c793e7eb7162f95ad95fd9901c10a11dfdfb8420b52f11035da948a]
08:08:48  module.taskflow.docker_container.lb: Destruction complete after 1s
08:08:48  module.taskflow.docker_container.app[0]: Destroying...
[id=43283eb6b8bec83b95b76f629de24c564354520a05894b4afb63041606107d02]
08:08:48  module.taskflow.docker_container.app[1]: Destroying...
[id=4b713bbe21f1b6302349b069abef5442fb67da39542aab6055b87899710c9fd9]
08:08:48  module.taskflow.docker_container.app[2]: Destroying...
[id=62eeb6f7f2dfd81e8b73441e72b9426b11ab979ba55b2fb0e26ddfb0e044bfc6]
08:08:49  module.taskflow.docker_container.app[1]: Destruction complete after 1s
08:08:49  module.taskflow.docker_container.app[2]: Destruction complete after 1s
08:08:49  module.taskflow.docker_container.app[0]: Destruction complete after 1s
08:08:49  module.taskflow.docker_network.this: Destroying... [id=e96ed7b158dbde3810206eb9f9866832abf33527f04892b02504a8146a38be39]
08:08:49  module.taskflow.docker_volume.data: Destroying... [id=taskflow-production-data]
08:08:51  module.taskflow.docker_volume.data: Destruction complete after 2s
08:08:51  module.taskflow.docker_volume.data: Creating...
08:08:51  module.taskflow.docker_volume.data: Creation complete after 0s [id=taskflow-production-data]
08:08:51  module.taskflow.docker_network.this: Destruction complete after 3s
08:08:51  module.taskflow.docker_network.this: Creating
```

Figure 28: Deploy to Production: Terraform replaces the containers with build 3

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Smoke Test Production)
[Pipeline] sh
08:09:05  + seq 1 30
08:09:05  + curl -fs http://host.docker.internal:8082/api/health
08:09:05  + break
08:09:05  + echo --- http://host.docker.internal:8082/api/info
08:09:05  --- http://host.docker.internal:8082/api/info
08:09:05  + curl -fsS http://host.docker.internal:8082/api/info
08:09:05  + tee smoke.json
08:09:05
{"app":"TaskFlow","version":"1.1.0","build":"3","commit":"1740bf6","environment":"production","instance":"2887cac227bb","startedAt"
:"2026-10-30T08:08:55.549Z"}+ echo
08:09:05
08:09:05  + grep -q "build":"3" smoke.json
08:09:05  + echo --- requests through the load balancer
08:09:05  --- requests through the load balancer
08:09:05  + curl -fsS http://host.docker.internal:8082/api/health
```

Figure 29: Smoke test of production

9.5 Build Result

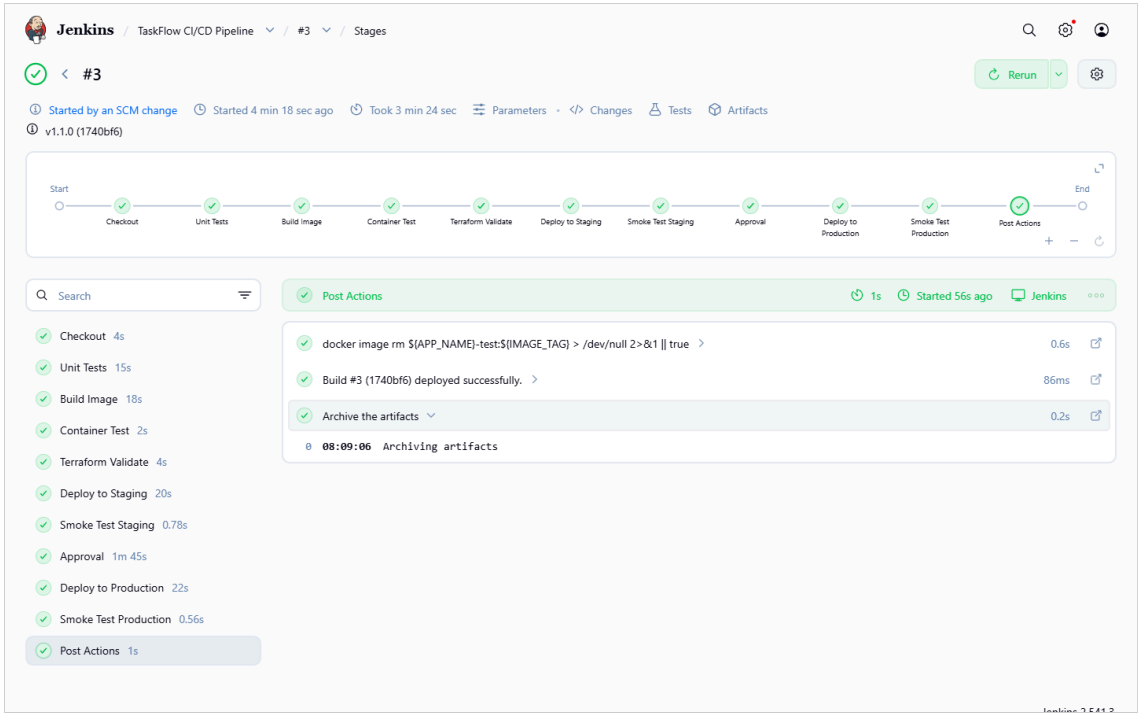


Figure 30: Pipeline graph of build #3: all stages successful

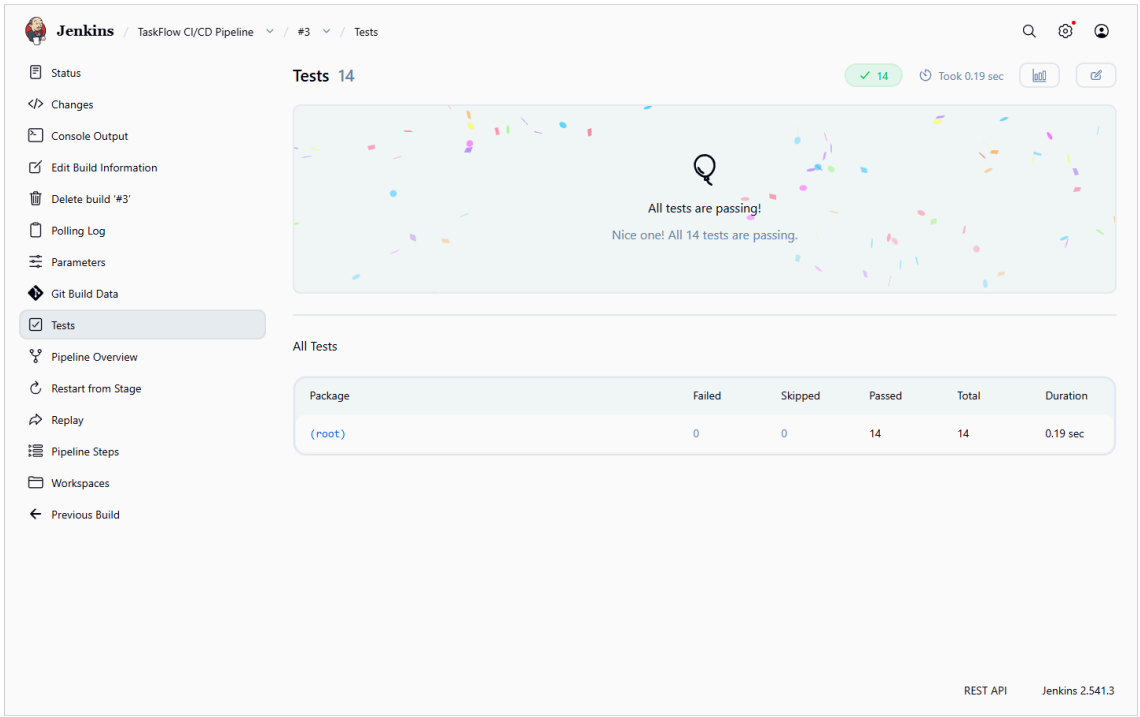


Figure 31: Test results published by the junit step: 14 tests, 0 failures



Figure 32: Archived artifacts: staging and production Terraform plans and the test report

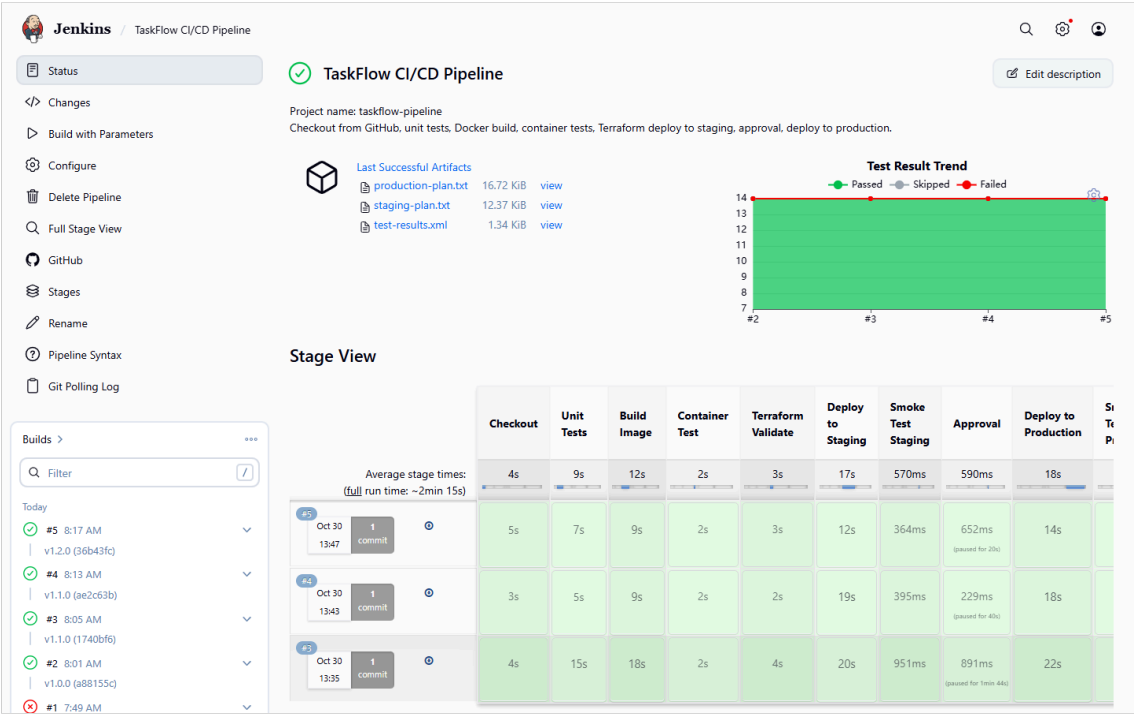


Figure 33: Job page: stage view, test trend and build history

9.6 Data Persistence Across Deployments

While checking the Terraform plan in the console of build #3, it showed **6 to add, 6 to destroy** for production, which included the data volume and the network. The cause is explained in section 11. After the fix (build #4), a task named "Customer feedback form" was created in production, and version 1.2.0 (keyboard shortcuts N and /) was pushed. Build #5 was started by the SCM change and deployed the new version:

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Approval)
[Pipeline] timeout
08:18:27  Timeout set to expire in 15 min
[Pipeline] {
[Pipeline] input
08:18:27  Staging is healthy. Deploy build #5 to production?
08:18:27  Deploy or Abort
08:18:48  Approved by Abhay Maurya
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Production)
[Pipeline] dir
08:18:48  Running in /var/jenkins_home/workspace/taskflow-pipeline/terraform
[Pipeline] {
[Pipeline] sh
08:18:48  + terraform workspace select -or-create production
08:18:48  Switched to workspace "production".
[Pipeline] sh
08:18:48  + terraform plan -no-color -out=production.tfplan -var-file=environments/production.tfvars -var image_tag=5 -var
app_version=1.2.0 -var build_number=5 -var git_commit=36b43fc
08:18:49  module.taskflow.data.docker_image.app: Reading...
08:18:49  module.taskflow.docker_volume.data: Refreshing state... [id=taskflow-production-data]
08:18:49  module.taskflow.docker_network.this: Refreshing state...
[id=df4dd0a501a702f5f9f8a384a72494d93c1bcd0ffe63beeaf1e0cd2895d1587f]
```

Figure 34: Build #5 plan for production: only the 3 app containers and the load balancer are replaced; the volume and network are kept

After build #5 the production board shows version 1.2.0, build #5, and the task created before the deployment is still there:

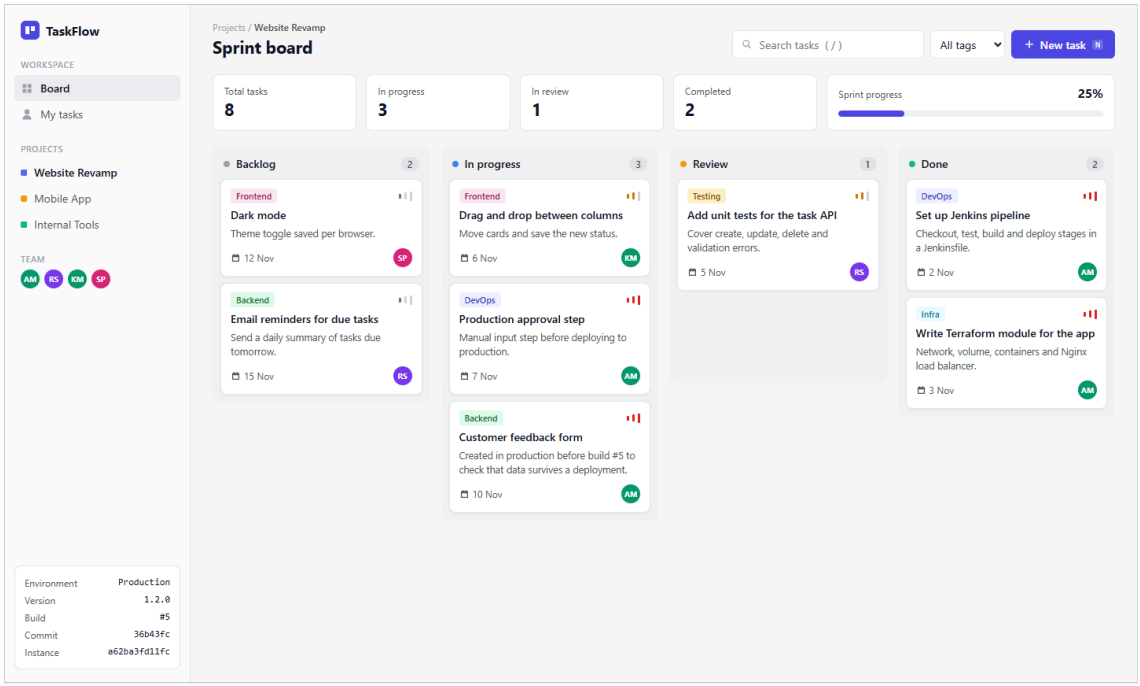


Figure 35: Production after build #5: version 1.2.0, and the "Customer feedback form" task survived the deployment

The deployed containers on the host, created and labelled by Terraform:

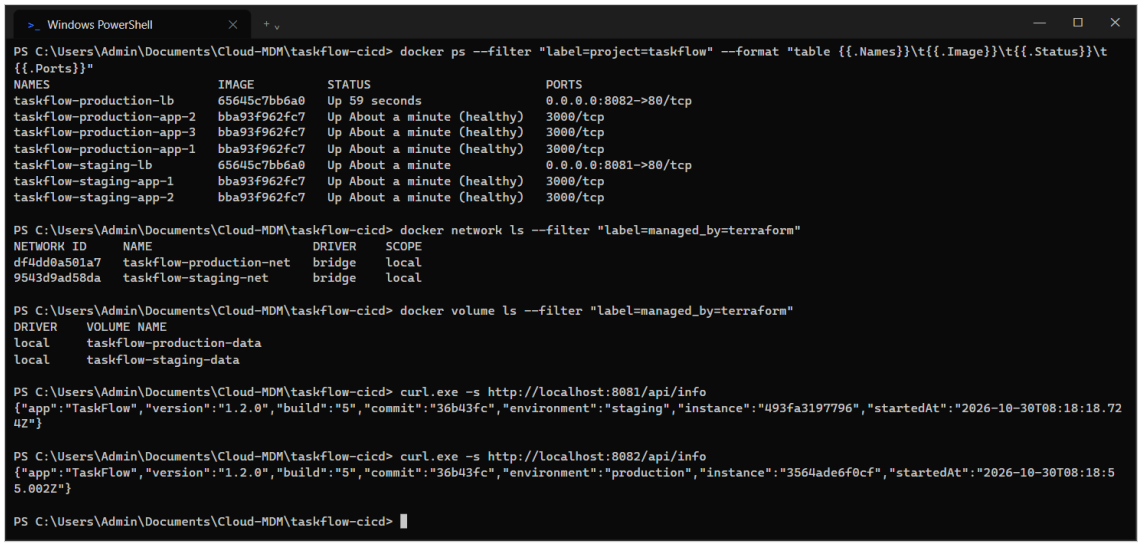


Figure 36: Staging and production containers, networks and volumes after build #5

10. TESTING AND RESULTS

10.1 Test Cases

Test ID	Description	Command / Method	Observed Result	Status
TC-01	Application tests	npm test / Unit Tests stage	14 passed, 0 failed	Pass
TC-02	Jenkins configured as code	docker compose up	Job created, no setup wizard	Pass
TC-03	Repository integration	git push, Poll SCM	Build started by an SCM change	Pass
TC-04	Image build	Build Image stage	taskflow:3 built with build args	Pass
TC-05	Container test	Container Test stage	API answered with build 3	Pass
TC-06	Terraform validation	fmt -check, validate	Configuration is valid	Pass
TC-07	Terraform by hand	init, plan, apply (dev)	5 resources created	Pass
TC-08	Reusable config	-var replicas=2	Second container added	Pass
TC-09	Destroy	terraform destroy	6 resources destroyed	Pass
TC-10	Deploy to staging	Deploy to Staging stage	2 replicas, build 3 served	Pass
TC-11	Approval gate	input step	Production waited for approval	Pass
TC-12	Deploy to production	Deploy to Production stage	3 replicas, build 3 served	Pass
TC-13	Data persistence	Build #5 after the fix	Volume kept, task still present	Pass

10.2 Build History

Build	Trigger	Change	Result
#1	Manual (Build Now)	First build of the job	Failed at checkout (temporary DNS error)
#2	Manual (Build Now)	Initial version 1.0.0	Success, staging and production deployed
#3	SCM change	Version 1.1.0, single checkout	Success (205 s including approval)
#4	SCM change	Terraform fix for volume and network labels	Success
#5	SCM change	Version 1.2.0, keyboard shortcuts	Success, data kept across the deployment

10.3 Summary of Results

- Jenkins was set up completely as code and integrated with the GitHub repository.
- Commits pushed to GitHub started the pipeline automatically through Poll SCM.
- Every successful build ran 14 tests, built a versioned image and tested the container before deployment.
- Terraform created staging and production from one reusable module, with separate workspaces.
- Production deployments waited for manual approval and were verified by smoke tests.
- Version 1.0.0, 1.1.0 and 1.2.0 were released through the pipeline without any manual deployment step.

11. CHALLENGES FACED

1. Data volume recreated on every deployment. The Terraform plan in the console of build #3 showed that the data volume and the network were destroyed and created again in every deployment, so all tasks were lost. The reason was that the build number was added as a label to every resource, and Docker cannot change the labels of an existing network or volume, so Terraform has to replace them. The fix was to keep only stable labels (project, environment, managed_by) on the network and volume and to add the build label only to the containers (container_labels). Build #4 removed the old label once, and build #5 showed that the volume is kept and the data survives.

2. Temporary network failure. Build #1 failed during checkout with "Could not resolve host: github.com". The Jenkins configuration was correct; the next build worked once the network recovered. This shows why a pipeline should stop at the first failing step.

3. Code checked out twice. The console of build #2 showed a "Declarative: Checkout SCM" step before my own Checkout stage. Adding skipDefaultCheckout(true) to the options removed the extra checkout.

4. Provider name in the module. The first terraform init failed because the module did not declare its provider, so Terraform looked for hashicorp/docker. Adding a versions.tf with kreuzwerker/docker to the module fixed it.

5. Running the test folder on Node.js 22. node --test test/ treats the folder name as a file in newer Node.js versions. Using node --test without an argument lets Node.js find the test files itself.

12. SECURITY AND BEST PRACTICES

- **No secrets in Git:** the Jenkins admin password is in jenkins/.env, which is in .gitignore; only .env.example is committed.

- **Least privilege for viewers:** anonymous users can only view builds; running builds and approving needs a login.
- **Approval before production:** production is never changed without a person approving the release.
- **Same artifact everywhere:** the image tested in the pipeline is the image deployed to staging and production.
- **Reviewed changes:** Terraform applies the saved plan file, and the plan is archived with the build.
- **Pinned versions:** Terraform and provider versions are pinned and the lock file is committed.
- **Non-root application:** the TaskFlow container runs as the node user with a memory limit and a health check.
- **Limitation:** the Jenkins container runs as root to use the Docker socket. This is acceptable for a local lab, but a production setup should use separate build agents.

13. FUTURE ENHANCEMENTS

1. **GitHub webhooks** instead of polling, once Jenkins has a public URL.
2. **Cloud infrastructure:** use the same structure with the AWS or Azure provider to create virtual machines or a managed container service.
3. **Remote state:** store Terraform state in S3 or Terraform Cloud with locking.
4. **Container registry:** push images to Docker Hub or GitHub Container Registry.
5. **Zero-downtime deployment:** blue-green or rolling replacement of containers.
6. **Build agents:** run builds on separate Jenkins agents instead of the controller.
7. **Notifications:** send build results to e-mail or Slack.
8. **Code quality:** add linting and test coverage reports to the pipeline.

14. CONCLUSION

This assignment implemented a complete CI/CD workflow for the TaskFlow application with Jenkins and Terraform. Jenkins was configured entirely as code and connected to the GitHub repository. Every commit pushed to the main branch was tested, built into a versioned Docker image, tested again as a running container and deployed to staging with Terraform. After a manual approval the same image was deployed to production, and smoke tests confirmed each deployment.

The Terraform configuration is structured into a root module, a reusable webapp module and per-environment variable files, with validation rules, outputs and separate workspaces. It was used by hand for the dev environment and by the pipeline for staging and production. Reading the Terraform plan in the pipeline log also revealed a real problem, the data volume being recreated on every deployment, which was fixed and verified in a later build.

The project gave practical experience with Jenkins pipelines, Configuration as Code, repository integration, automated testing, Infrastructure as Code, Terraform state and workspaces, and safe release practices such as approval gates and smoke tests.

❖ GitHub Repository

The source code, Jenkinsfile, Terraform configuration, Jenkins setup files and screenshots are available on GitHub:

GitHub Repository Link:
https://github.com/Abhxym/taskflow-cicd

Repository Contents:

- Jenkinsfile - the CI/CD pipeline.

- terraform/ - root module, webapp module and environment variable files.
- jenkins/ - Jenkins Dockerfile, plugins, Configuration as Code and docker-compose file.
- app/, public/, test/ - TaskFlow source code and tests.
- Dockerfile - multi-stage image with the test stage.
- README.md - setup and run instructions.
- docs/screenshots/ - execution evidence.

REFERENCES

1. Jenkins Documentation. Pipeline. Available at: https://www.jenkins.io/doc/book/pipeline/
2. Jenkins Documentation. Pipeline Syntax. Available at: https://www.jenkins.io/doc/book/pipeline/syntax/
3. Jenkins Configuration as Code Plugin. Available at: https://plugins.jenkins.io/configuration-as-code/
4. Job DSL Plugin. Available at: https://plugins.jenkins.io/job-dsl/
5. Terraform Documentation. Available at: https://developer.hashicorp.com/terraform/docs
6. Terraform Language: Modules. Available at: https://developer.hashicorp.com/terraform/language/modules
7. Terraform Workspaces. Available at: https://developer.hashicorp.com/terraform/language/state/workspaces
8. Docker Provider for Terraform (kreuzwerker/docker). Available at: https://registry.terraform.io/providers/kreuzwerker/docker/latest/docs
9. Docker Documentation. Multi-stage builds. Available at: https://docs.docker.com/build/building/multi-stage/
10. Node.js Documentation. Test runner. Available at: https://nodejs.org/api/test.html